

Explainable Artificial Intelligence Model for Predictive Maintenance in Smart Agricultural Facilities

ABSTRACT

In modern smart agricultural facilities, the continuous operation of equipment such as pumps, irrigation systems, and climate control devices is essential for maintaining productivity and sustainability. However, unexpected equipment failures often lead to costly downtime, reduced yields, and higher maintenance costs. Traditional maintenance practices, including reactive and preventive approaches, are limited in their ability to anticipate failures, leaving farmers with inefficient resource utilization and unpredictable risks. This creates a pressing need for predictive maintenance solutions that can leverage real-time data and advanced analytics to ensure timely interventions.

To address this problem, this project proposes an **Explainable Artificial Intelligence (XAI) model for predictive maintenance in smart agricultural facilities**. The system integrates multiple machine learning and deep learning algorithms, including LSTM, BiLSTM, ANN, and LightGBM, to analyze sensor data and predict potential failures. The Django-based web platform allows administrators to upload datasets, preprocess data, train models, evaluate performance, and visualize results using confusion matrices and comparative performance charts. Furthermore, the system supports prediction on new data, enabling stakeholders to make proactive decisions. By combining predictive accuracy with explainability, the solution empowers users to not only forecast failures but also understand model outputs, thereby fostering trust in AI-driven maintenance strategies.

Keywords: Predictive Maintenance, Smart Agriculture, Explainable AI, LSTM, BiLSTM, ANN, LightGBM, Django.

INTRODUCTION

The advancement of smart agriculture has led to the integration of automation, sensors, and data-driven decision-making to enhance productivity and sustainability in farming practices. In modern agricultural facilities, critical equipment such as pumps, motors, and irrigation systems operate continuously to maintain environmental conditions essential for crop growth. However, the unexpected breakdown of such equipment can disrupt operations, leading to crop losses, resource wastage, and increased operational costs. Traditional maintenance practices—reactive or preventive—are often inefficient, as they either address problems only after failure or rely on scheduled servicing that may not correspond to the actual health of machinery. This highlights the urgent need for **predictive maintenance systems** that can proactively detect early signs of malfunction and recommend timely interventions.

The dataset used in this study, *agri_predictive_maintenance.csv*, contains 50,000 records with 9 input features and a target variable. Features such as **temperature, humidity, soil moisture, vibration levels, motor current, pump status, last maintenance days, and error codes** provide valuable insights into the operational conditions of agricultural equipment. The target variable, **maintenance_required**, indicates whether machinery requires servicing, enabling the development of supervised learning models. By leveraging this dataset, predictive models can be trained to identify hidden patterns associated with equipment failures and maintenance needs.

To build an effective solution, this project integrates **Explainable Artificial Intelligence (XAI) techniques** with machine learning and deep learning algorithms, including LSTM, BiLSTM, ANN, and LightGBM. The objective is not only to achieve high predictive accuracy but also to ensure that the decision-making process remains interpretable for agricultural stakeholders. The proposed system, developed using a Django web platform, allows seamless dataset management, model training, performance evaluation, and real-time prediction. This approach empowers farmers and facility managers to adopt proactive maintenance strategies, reducing downtime, optimizing resource usage, and fostering trust in AI-based decision support.

LITERATURE SURVEY

Kisten, M., Ezugwu, A. E.-S., & Olusanya, M. O. (2024).

This study proposes an explainable artificial intelligence (XAI) framework specifically designed for predictive maintenance in smart agricultural facilities. The authors highlight the limitations of traditional predictive models that function as black boxes, creating trust issues among stakeholders. Their model integrates sensor-based monitoring with machine learning classifiers while embedding interpretability mechanisms such as SHAP values and feature attribution to explain predictions. The framework demonstrates high accuracy in identifying equipment failures, including pumps and motors, while offering transparent decision paths that help facility managers understand why a machine is predicted to require maintenance. One of the key contributions is the balance achieved between model performance and interpretability, ensuring that predictive maintenance systems are not only accurate but also user-friendly for non-technical agricultural operators. This research underscores the importance of adopting explainable AI to foster trust, reliability, and actionable insights in smart farming environments.

Molęda, M. (2023).

Molęda presents a comprehensive review that traces the evolution of maintenance strategies from corrective and preventive approaches to predictive maintenance. The paper emphasizes how Industry 4.0 technologies, including the Internet of Things (IoT), big data analytics, and machine learning, have enabled predictive models to surpass traditional methods. The author systematically categorizes predictive maintenance frameworks across industries, including agriculture, manufacturing, and transportation. A central theme is the identification of data-driven approaches that leverage sensor streams to forecast equipment failures before they occur. The review also outlines key challenges such as data quality issues, integration of heterogeneous sensor systems, and the need for explainable AI to enhance transparency in predictions. By consolidating best practices, research gaps, and future directions, this work serves as a foundational reference for researchers aiming to apply predictive maintenance techniques in domains like smart agriculture, where machinery health is critical to sustained operations.

Shaif, A. (2021).

Shaif's master's thesis focuses on applying machine learning techniques to predictive maintenance in smart agriculture. The research leverages real-time sensor data from

agricultural equipment, including vibration levels, motor current, and environmental variables, to predict maintenance requirements. Several supervised learning algorithms are explored, including Random Forests, Support Vector Machines, and neural networks, with performance comparisons based on accuracy, precision, and recall. Importantly, the thesis stresses the role of feature engineering in improving model effectiveness, such as deriving key indicators from raw vibration signals. The study also highlights practical constraints, including the variability of environmental conditions and data imbalance between failure and normal operation cases. By developing and validating predictive models, the thesis demonstrates the feasibility of using machine learning to reduce downtime and operational costs in agriculture. This work contributes a practical academic perspective to the field, bridging theoretical machine learning methods with agricultural maintenance applications.

Hrast Essenfelder, A., Toreti, A., & Seguni, L. (2025).

This paper explores the use of explainable AI models to detect climate hazards relevant to agriculture, such as heat stress, drought, and extreme precipitation. The authors employ expert-driven AI approaches that combine machine learning models with domain-specific agricultural knowledge. Unlike purely data-driven models, the inclusion of expert knowledge ensures that the predictions are not only accurate but also interpretable and actionable for decision-makers. The framework incorporates interpretability methods to show how input features—such as temperature anomalies or soil moisture levels—affect model outputs. The study demonstrates that such explainable models can enhance early warning systems and assist in mitigating risks to crops and infrastructure in agricultural facilities. While not focused exclusively on predictive maintenance, the methodology reinforces the importance of transparency in AI applications within agriculture, aligning closely with the broader goal of making smart agricultural systems trustworthy, resilient, and adaptive to environmental uncertainties.

Abdallah, M., et al. (2021).

Abdallah and colleagues investigate anomaly detection in IoT-based agriculture and manufacturing systems using transfer learning. Their approach acknowledges the challenge of limited labeled failure data in predictive maintenance tasks. To overcome this, the authors apply transfer learning by adapting models trained on one domain (e.g., manufacturing) to another (e.g., agriculture). The proposed framework leverages deep learning architectures capable of extracting features from raw sensor data, such as vibration and current measurements, and applies anomaly detection techniques to identify unusual patterns that may indicate equipment

faults. The novelty lies in reducing the need for extensive retraining while still achieving high accuracy across multiple domains. The research highlights the potential of transfer learning to make predictive maintenance scalable and cost-effective, especially in smart agriculture where datasets are often sparse. The study underscores the critical role of data reusability and adaptability in building robust anomaly detection systems.

Ngo, Q. H., Kechadi, T., & Le-Khac, N.-A. (2022).

This paper introduces OAK4XAI, a framework aimed at delivering out-of-the-box explainable artificial intelligence solutions for digital agriculture. The authors argue that while AI adoption in agriculture has accelerated, the lack of transparency limits stakeholders' trust in such systems. OAK4XAI addresses this by integrating interpretability modules that explain predictions from machine learning models used in agricultural tasks, such as yield estimation, soil classification, and predictive maintenance. The system provides end-users with visual and textual explanations, making the decision-making process more transparent. Importantly, the framework is designed to be modular and adaptable, enabling seamless deployment across various agricultural AI applications without extensive customization. By focusing on generalizability and user-friendliness, OAK4XAI contributes to the democratization of explainable AI in agriculture. This work strengthens the foundation for integrating interpretability in predictive maintenance systems, ensuring that stakeholders not only receive predictions but also understand the rationale behind them.

Turgut, O., Kok, I., & Ozdemir, S. (2024).

Turgut and colleagues propose AgroXAI, an explainable AI-driven crop recommendation system tailored for Agriculture 4.0. While primarily focused on crop selection, the study introduces key XAI principles that can be applied to predictive maintenance. The system integrates machine learning algorithms with interpretability tools like SHAP to provide transparent recommendations about which crops are best suited under specific environmental and soil conditions. Farmers are presented with both the recommendation and an explanation of the factors influencing it, such as soil moisture or nutrient levels. This approach highlights the broader value of XAI in agriculture by demonstrating that transparency improves user trust and adoption of AI systems. Though not directly aimed at machinery maintenance, AgroXAI's methodology offers transferable lessons for predictive maintenance, particularly the importance of making complex AI decisions understandable and actionable for non-technical agricultural stakeholders.

EXISTING SYSTEM

In existing agricultural facilities, equipment maintenance is generally carried out using traditional approaches such as **corrective maintenance** and **preventive maintenance**. Corrective maintenance involves repairing or replacing equipment only after a failure occurs. While this method is straightforward, it often leads to unexpected downtime, costly disruptions, and crop damage. Preventive maintenance, on the other hand, follows fixed schedules, where machinery is serviced at regular intervals regardless of its actual condition. Although this reduces sudden breakdowns, it can result in unnecessary servicing, wasted resources, and increased operational costs. Additionally, some existing systems rely on simple threshold-based monitoring, where sensor values such as temperature, vibration, or motor current are compared against predefined limits. When thresholds are exceeded, alerts are triggered. However, this method lacks adaptability and cannot detect complex fault patterns. Overall, these approaches fail to incorporate advanced analytics or predictive intelligence, making them limited in reliability and efficiency for smart agricultural environments.

.

DISADVANTAGES

1. **Unplanned Downtime** – Corrective maintenance leads to sudden breakdowns, disrupting critical agricultural processes.
2. **Resource Wastage** – Preventive maintenance often results in unnecessary servicing when machinery is still in good condition.
3. **Low Accuracy** – Threshold-based systems are unable to capture complex fault interactions or gradual performance degradation.
4. **High Operational Cost** – Frequent manual inspections and excessive servicing increase maintenance expenses.
5. **Lack of Transparency** – Traditional methods do not provide insights into the underlying causes of failures, reducing trust in decision-making.

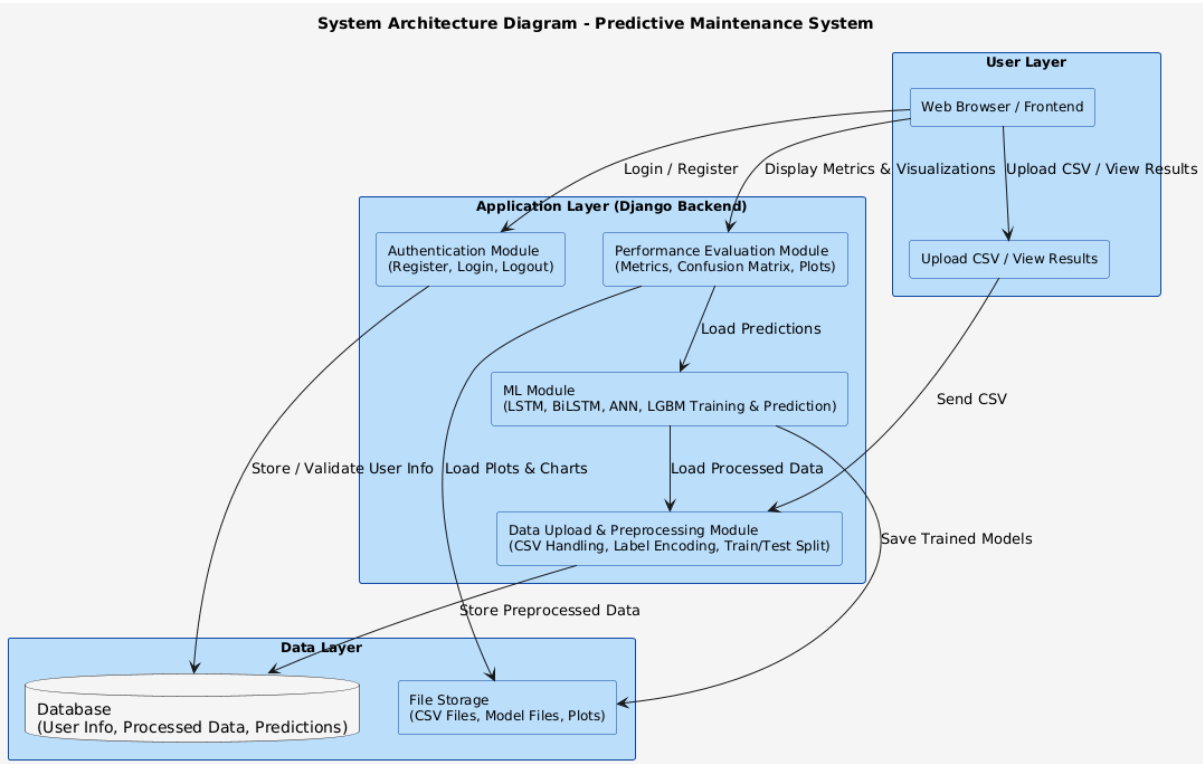
PROPOSED SYSTEM

The proposed system introduces an **Explainable Artificial Intelligence (XAI)–driven predictive maintenance model** tailored for smart agricultural facilities. Unlike traditional corrective or preventive methods, this system leverages real-time sensor data—such as temperature, humidity, soil moisture, vibration levels, motor current, pump status, and error codes—to forecast potential failures before they occur. Advanced machine learning and deep learning algorithms, including LSTM, BiLSTM, ANN, and LightGBM, are employed to detect hidden patterns in equipment behavior. The integration of **explainability techniques** (e.g., SHAP values) ensures that predictions are transparent, allowing stakeholders to understand why maintenance is required. Implemented through a Django-based web platform, the system supports dataset management, preprocessing, model training, performance visualization, and real-time fault prediction. This proactive and interpretable approach minimizes downtime, reduces unnecessary maintenance, and optimizes agricultural resource usage, creating a reliable decision-support tool for farmers and facility managers.

ADVANTAGES

1. **Reduced Downtime** – Predictive analytics anticipate failures early, preventing sudden breakdowns.
2. **Optimized Maintenance** – Equipment is serviced only when necessary, avoiding resource wastage.
3. **High Accuracy** – Advanced algorithms capture complex fault patterns beyond simple threshold monitoring.
4. **Cost Efficiency** – Lower maintenance costs are achieved through data-driven decision-making and minimized manual inspections.
5. **Transparency & Trust** – XAI provides clear explanations of predictions, helping stakeholders understand and trust AI-driven insights.

SYSTEM ARCHITECTURE



System Architecture

SYSTEM REQUIREMENTS

H/W SYSTEM CONFIGURATION:

- **RAM** - 4 GB (min)
- **Hard Disk** - 20 GB
- **Key Board** - Standard Windows Keyboard
- **Mouse** - Two or Three Button Mouse
- **Monitor** - SVGA

SOFTWARE REQUIREMENTS:

- **Operating system** : Windows 7 Ultimate.
- **Coding Language** : Python.
- **Back-End** : Django-ORM
- **Designing** : Html, css, javascript.
- **Data Base** : MySQL / (WAMP Server).

SYSTEM STUDY

FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

SYSTEM DESIGN

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Class diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram was capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

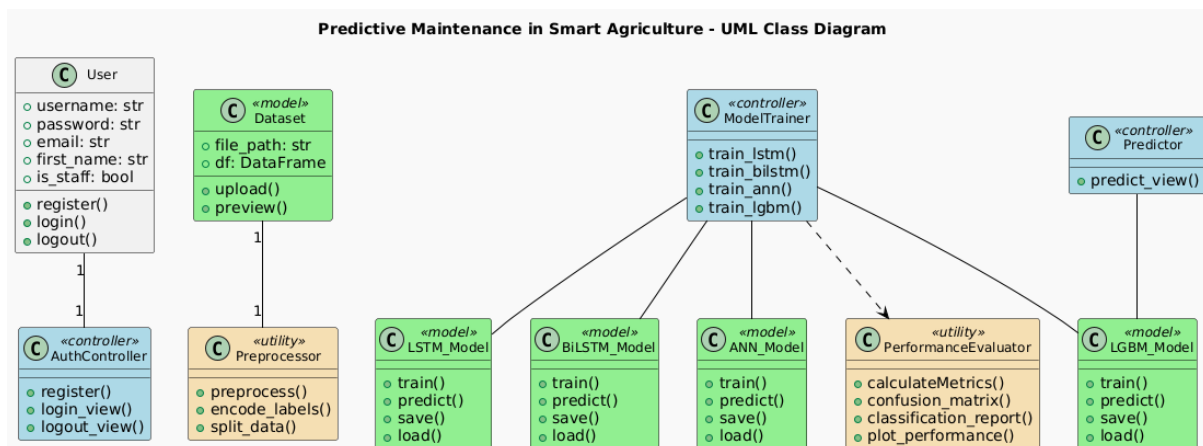


Figure-5.1: Class Diagram

Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows, as parallel vertical lines ("lifelines"), different processes or objects that live simultaneously, and as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

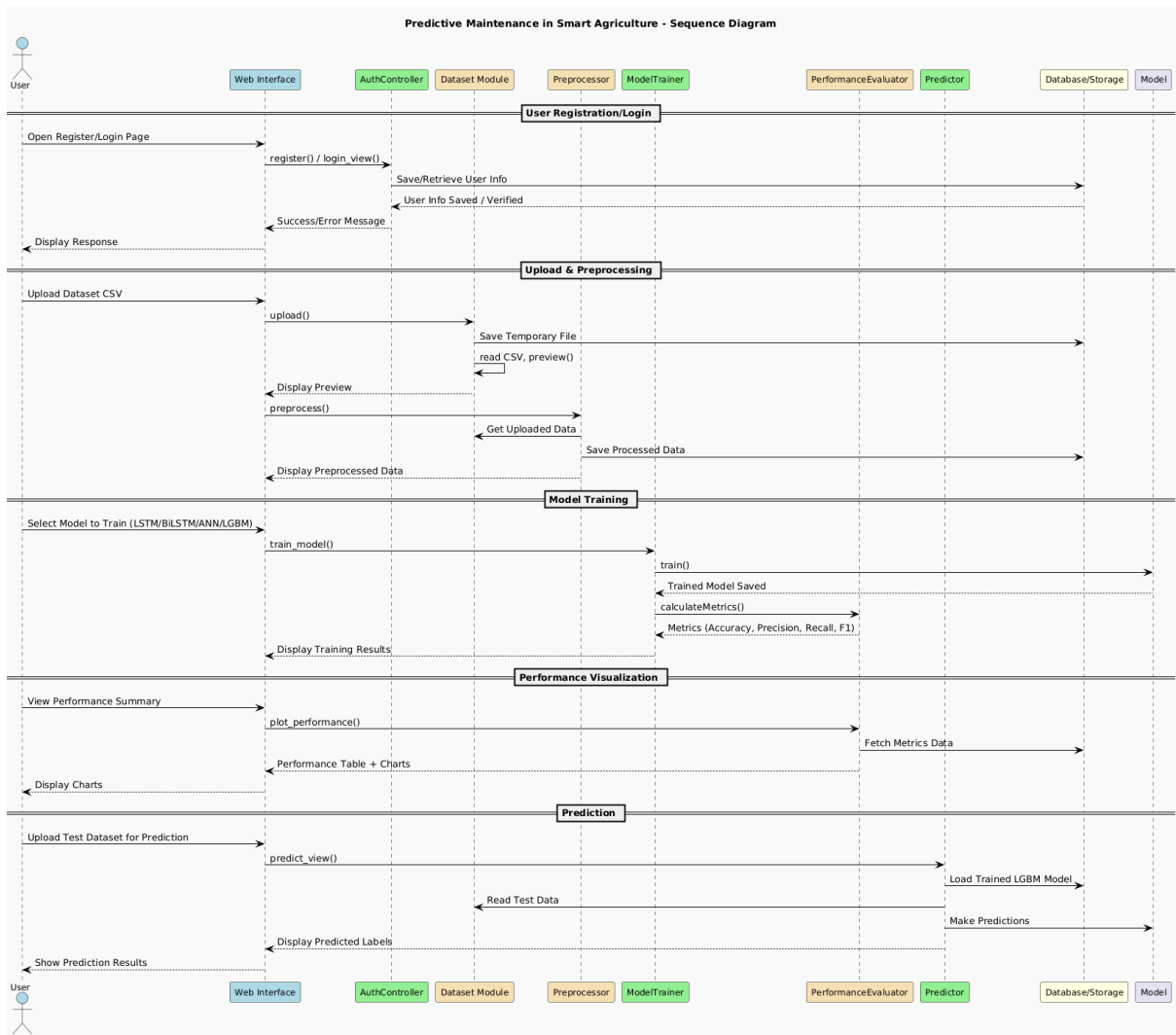


Figure-5.2: Sequence Diagram

Activity diagram

Activity diagrams are graphical representations of Workflows of stepwise activities and actions with support for choice, iteration, and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

Predictive Maintenance in Smart Agriculture - Activity Diagram

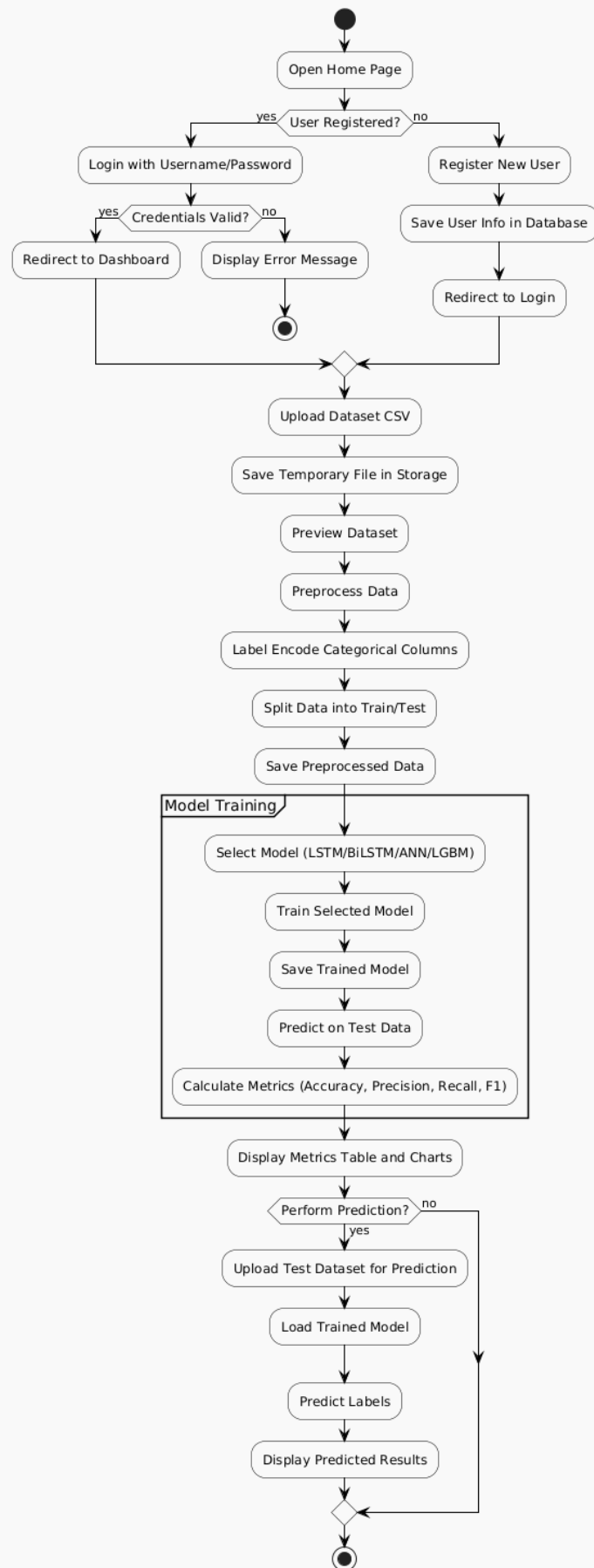


Figure-5.3: Activity Diagram

Data flow diagram

A data flow diagram (DFD) is a graphical representation of how data moves within an information system. It is a modeling technique used in system analysis and design to illustrate the flow of data between various processes, data stores, data sources, and data destinations within a system or between systems. Data flow diagrams are often used to depict the structure and behavior of a system, emphasizing the flow of data and the transformations it undergoes as it moves through the system.

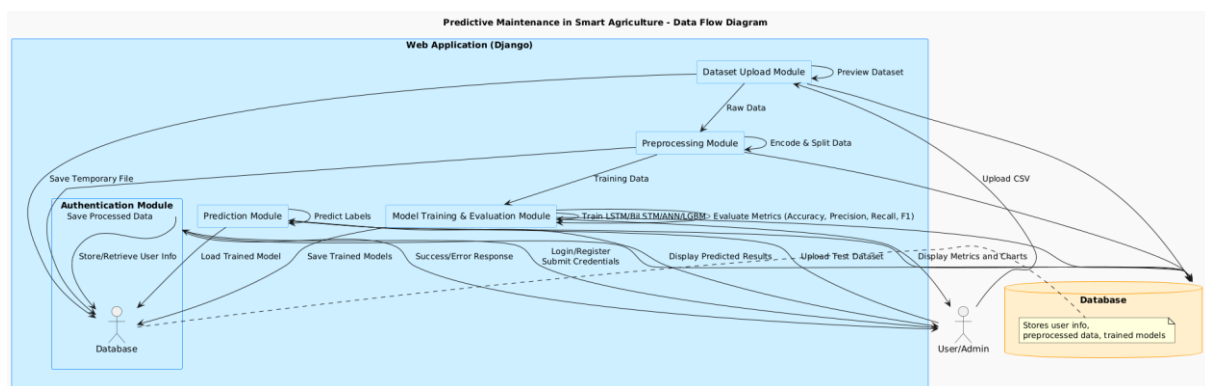


Figure-5.4: Dataflow Diagram

Component diagram: Component diagram describes the organization and wiring of the physical components in a system.

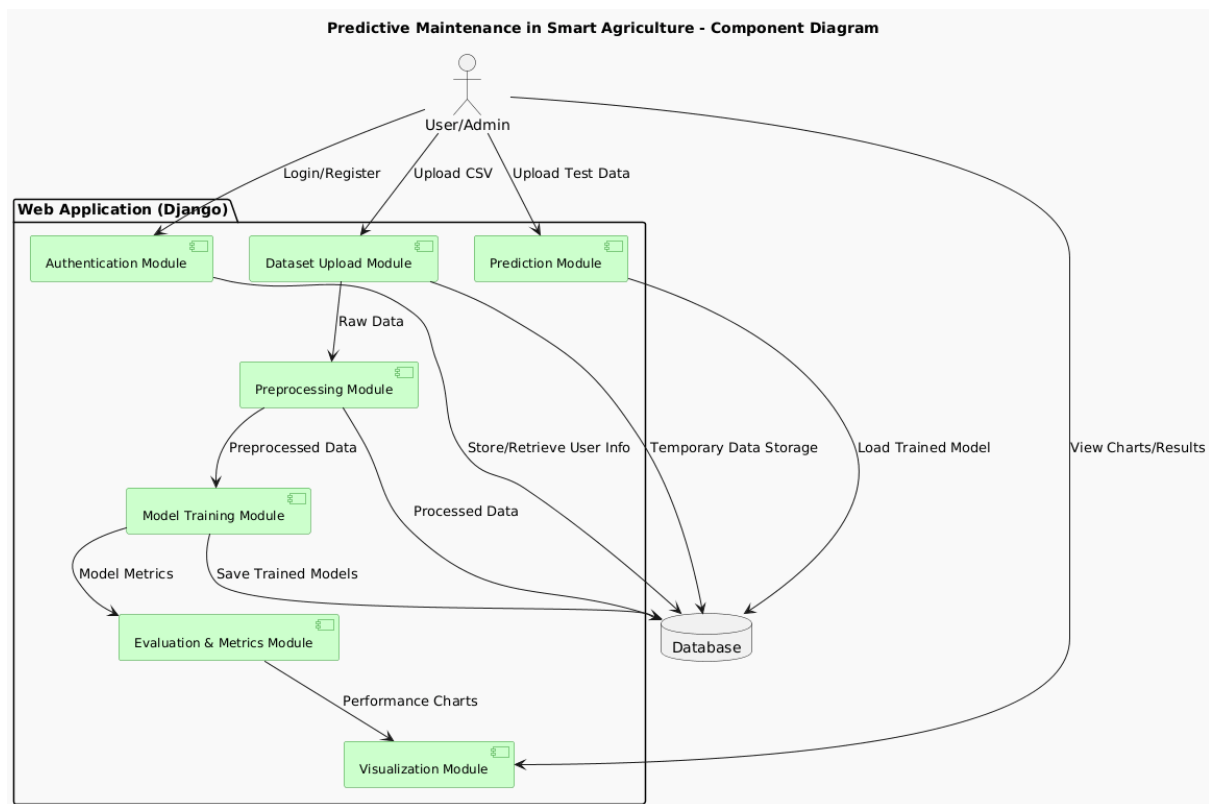


Figure-5.5: Component Diagram

Use Case diagram: A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

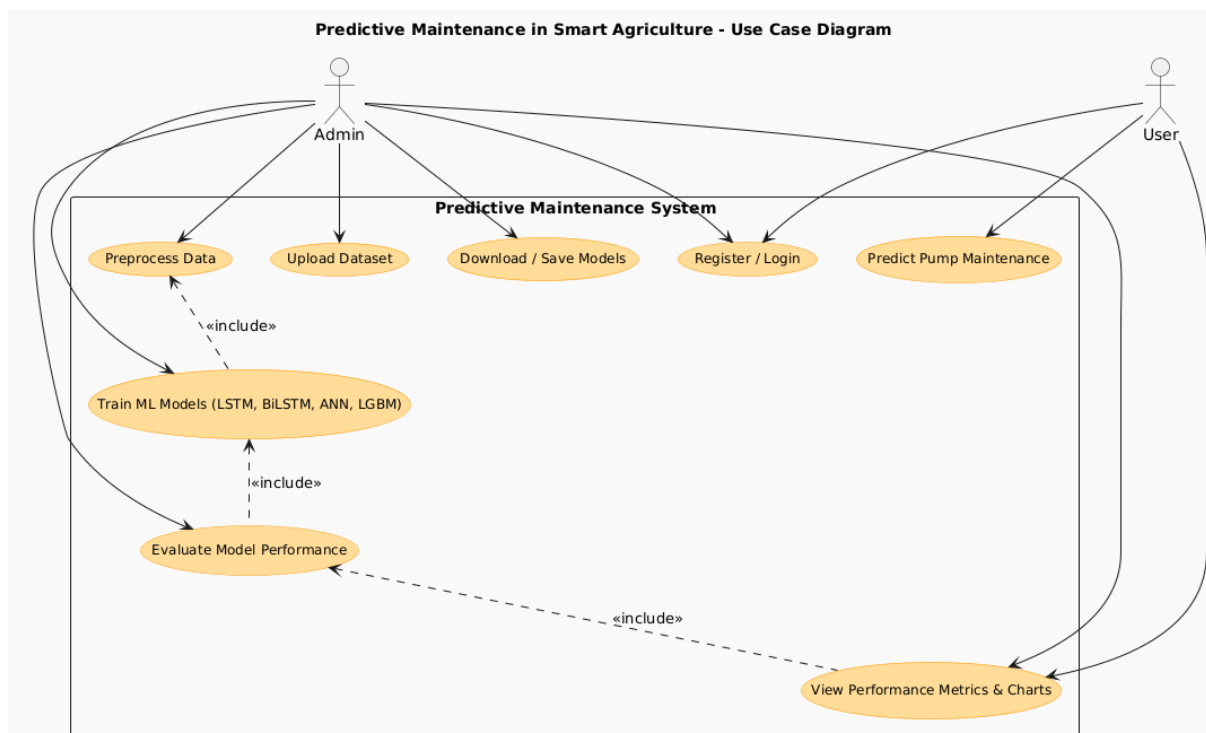


Figure-5.6: Use Case Diagram

Deployment Diagram:

A deployment diagram in UML illustrates the physical arrangement of hardware and software components in the system. It visualizes how different software artifacts, such as data processing scripts and model training components, are deployed across hardware nodes and interact with each other, providing insight into the system's infrastructure and deployment strategy.

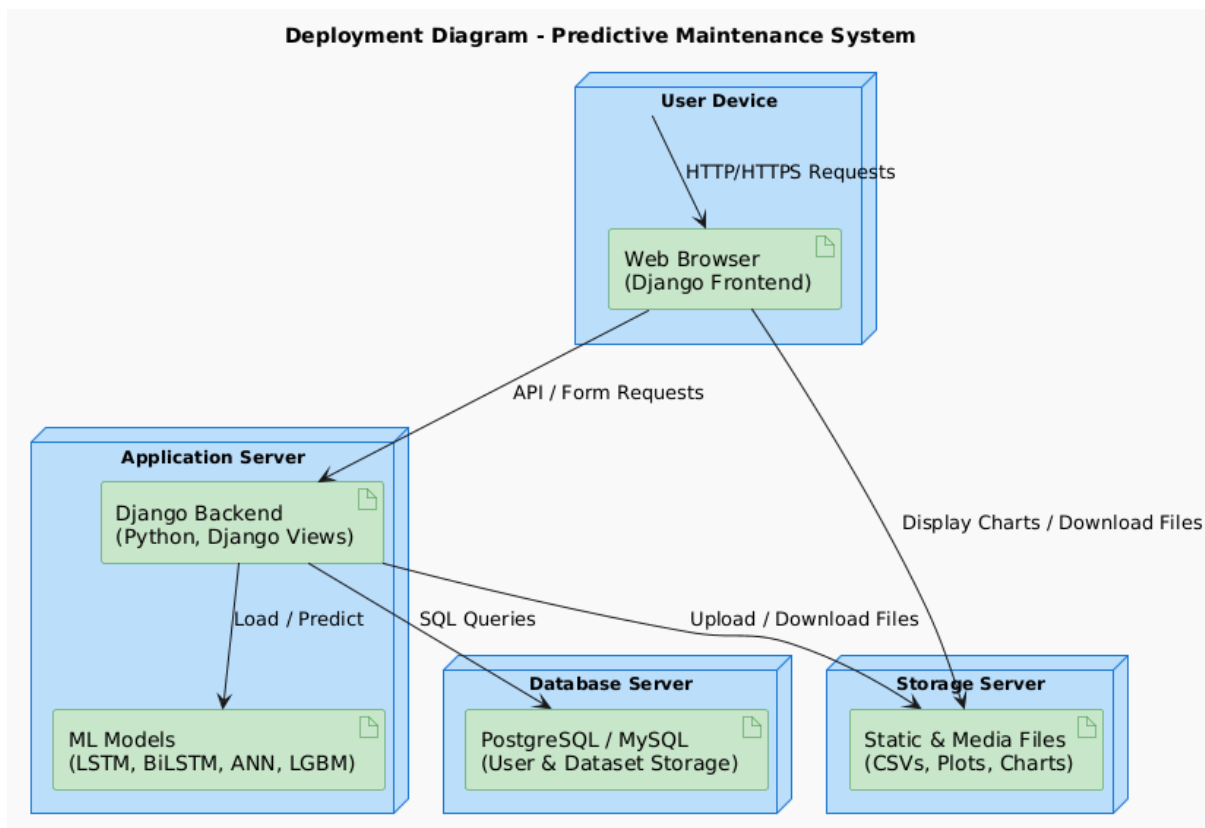


Figure-5.7: Deployment Diagram

Architectural Block Diagram

An architectural block diagram offers a high-level view of a system's structure, showcasing the main components and their interactions. It represents how major modules, such as data sources, processing units, and evaluation components, are organized and how they communicate with each other to accomplish the system's objectives. This diagram helps in understanding the overall design and flow of the system.

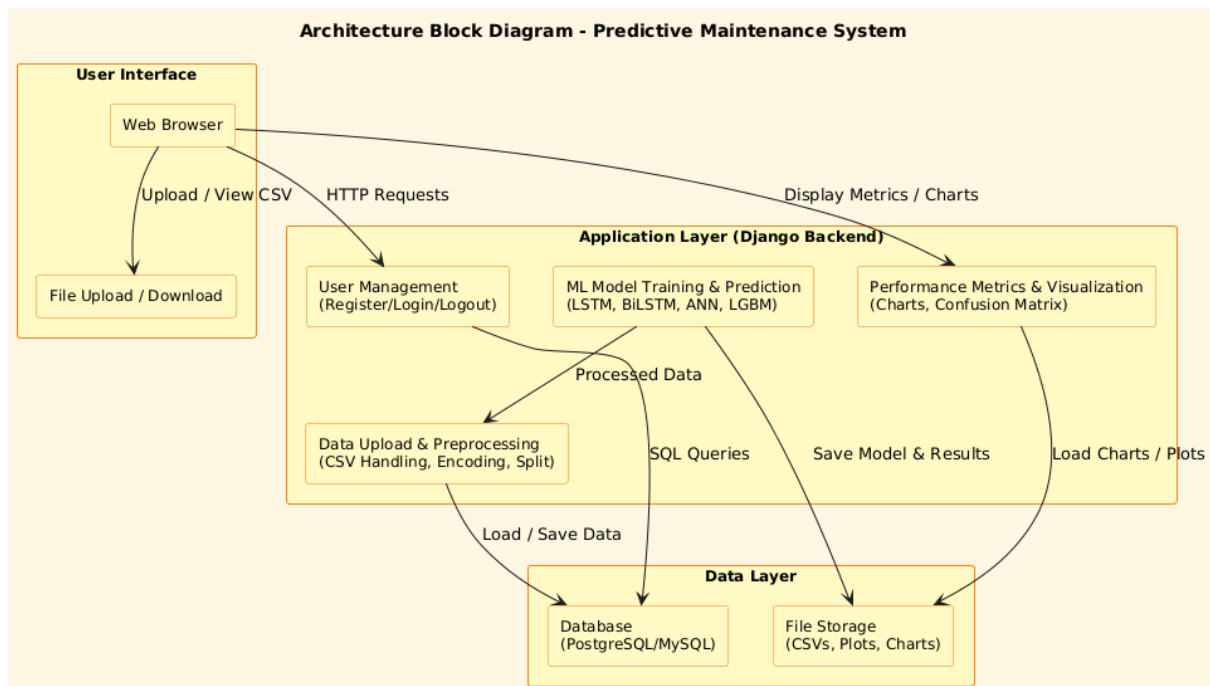


Figure 5.8: Architectural Block Diagram

SOFTWARE ENVIRONMENT

What is Python :-

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- [Machine Learning](#)
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it *contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more*. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you [download Python](#) for free, but you can also download its source code, make changes to it, and even distribute it. It comes with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and [machine learning](#), automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since [Python](#) is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python : -

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

Modules Used in Project

TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or

tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e., operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So, the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet [here](#). The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Cropped source tarball	Source release		68111671e5b2db4aef7b9ab010f09be	23017663	SIG
XZ compressed source tarball	Source release		d33e4aan66097051c2eca45ee3604803	17131432	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b467583da71a442cbalcee08e6	34898416	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773bf5e4a936b241f	28082845	SIG
Windows .help file	Windows		d63999573a2c56b2ac56cad6b47cd2	8131761	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9800c3cfd3ec0b9abe63184a4072ba2	7504291	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0ad76d9db3543a583e563400	2688368	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	2dc31c608b6d73ae6f53a3bd351b4bd2	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3bd18841879fda94113574139d8	6741626	SIG
Windows x86 executable installer	Windows		33cc802942a54446a3d6451476394789	25663848	SIG
Windows x86 web-based installer	Windows		1b670cfa5d311df82c30983ea371d87c	1324608	SIG

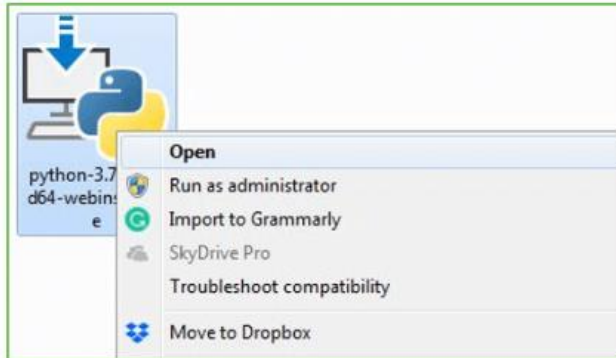
- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e., Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



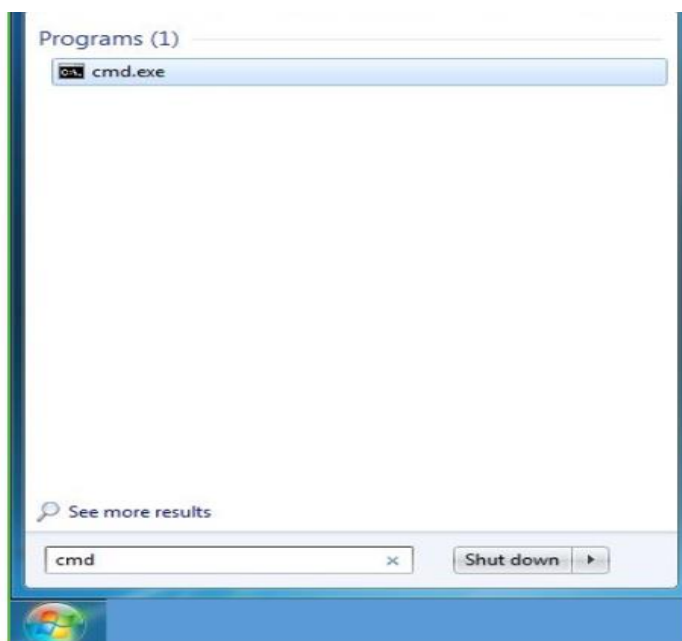
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

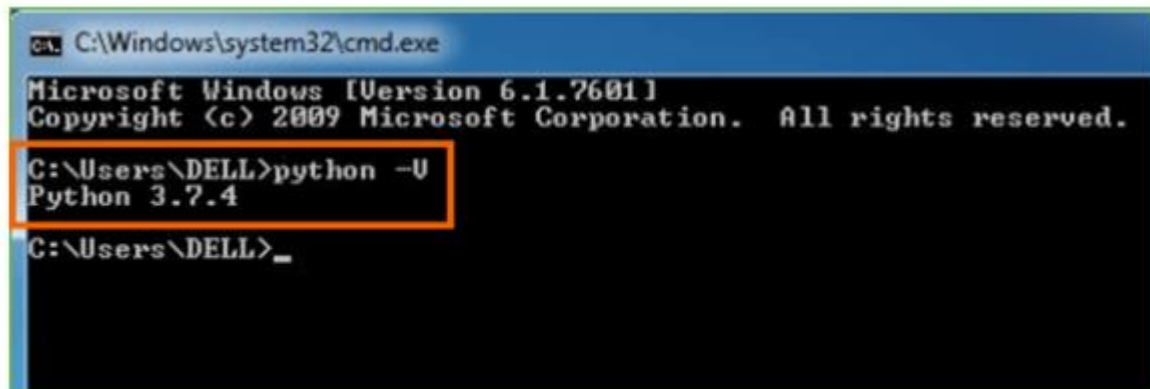
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python -V and press Enter.



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\DELL>python -V
Python 3.7.4

C:\Users\DELL>_
```

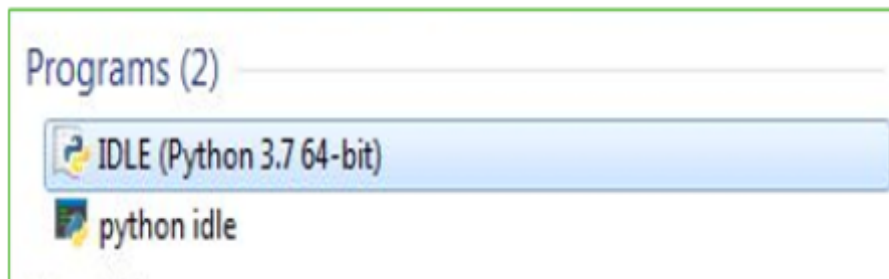
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

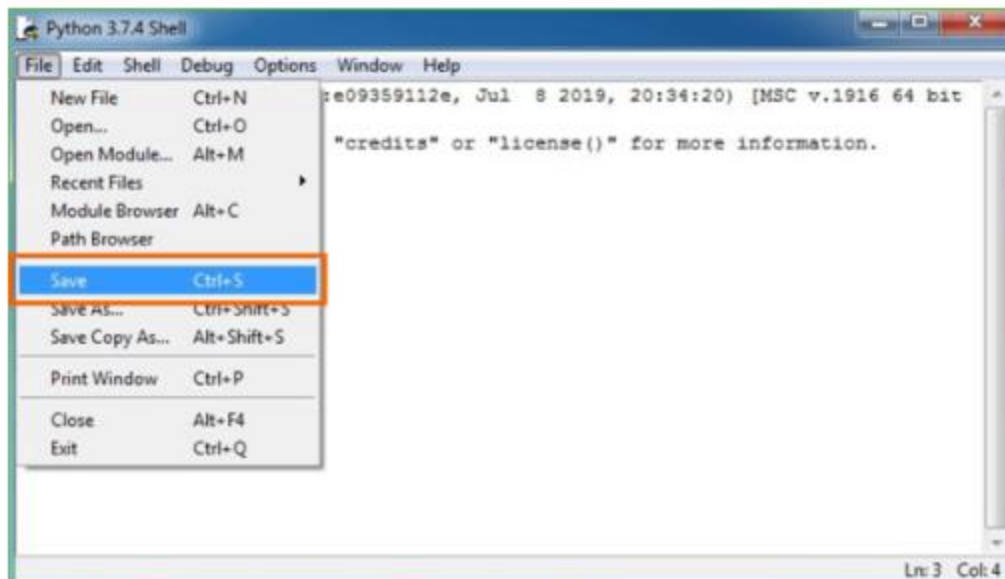
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



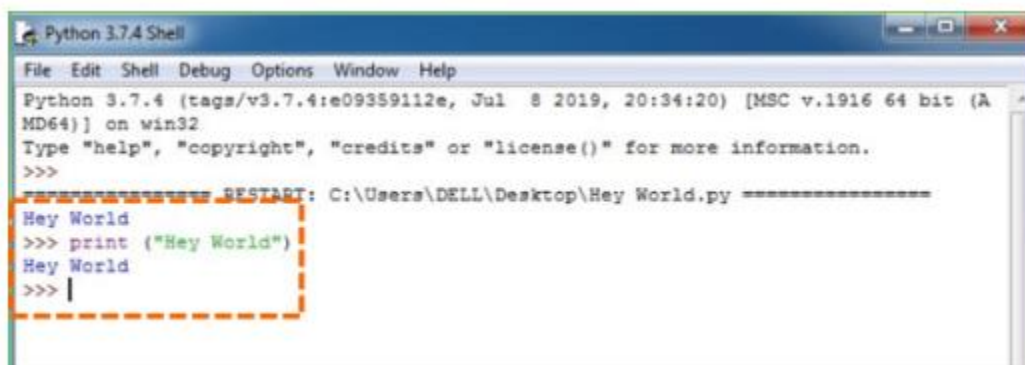
Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g., enter print ("Hey World") and Press Enter.



You will see that the command given is launched. With this, we end our tutorial on how to install Python. You have learned how to download python for windows into your respective operating system.

Note: Unlike Java, Python does not need semicolons at the end of the statements otherwise it won't work.

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction*. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML

—

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term “Machine Learning” in 1959 and defined it as a “Field of study that gives computers the capability to learn without being explicitly programmed”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to [Indeed](#), Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and an average base salary of \$146,085 per year.

But there is still a lot of doubt about what exactly is Machine Learning and how to start learning it? So this article deals with the Basics of Machine Learning and also the path you can follow to eventually become a full-fledged Machine Learning Engineer. Now let's get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection,

analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is [Python](#)! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as [Keras](#), [TensorFlow](#), [Scikit-learn](#), etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as [Fork Python](#) available Free on GeeksforGeeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.

- Reinforcement Learning – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As [ML algorithms](#) gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

[Machine Learning](#) is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

SYSTEM TEST

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

IMPLIMENTATION

Code.py:

```
from django.shortcuts import render,redirect  
  
from django.contrib.auth.models import User  
  
from django.contrib import messages  
  
from django.contrib.auth import login ,logout,authenticate  
  
from django.core.files.storage import default_storage  
  
import matplotlib.pyplot as plt  
  
import pandas as pd  
  
import os  
  
from tensorflow.keras.models import Model, load_model  
  
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D,  
Dropout, Flatten, BatchNormalization  
  
from xgboost import XGBClassifier  
  
import joblib  
  
from tensorflow.keras.layers import GRU  
  
from tensorflow.keras.optimizers import Adam  
  
from tensorflow.keras.callbacks import EarlyStopping,  
ReduceLROnPlateau  
  
from tensorflow.keras.models import Model, load_model  
  
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D,  
Dropout, Flatten, Dense, BatchNormalization
```

```
from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report, confusion_matrix

import numpy as np

import os

import matplotlib.pyplot as plt

import seaborn as sns

from tensorflow.keras.models import Model, load_model

from tensorflow.keras.layers import Input, Conv1D, Add,
BatchNormalization, Activation, MaxPooling1D, Flatten, Dense

from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.utils import to_categorical


# Required for plotting

import matplotlib.pyplot as plt

import seaborn as sns


# Declare label list (used in your classification report & confusion matrix)

Label = ["failure","normal"]
```

Create your views here.

def home(request):

return render(request,'Home.html')

from django.shortcuts import render

def about(request):

return render(request, 'about.html')

def register(request):

if request.method == 'POST':

First_Name = request.POST['name']

Email = request.POST['email']

username = request.POST['username']

password = request.POST['password']

confirmation_password = request.POST['cnfm_password']

select_user=request.POST['role']

if select_user=='admin':

admin=True

else:

admin=False

if password == confirmation_password:

```
    if User.objects.filter(username=username).exists():

        messages.error(request, 'Username already exists, please choose a
different one.')

        return redirect('register')

    else:

        if User.objects.filter(email=Email).exists():

            messages.error(request, 'Email already exists, please choose a
different one.')

            return redirect('register')

        else:

            user = User.objects.create_user(

                username=username,

                password=password,

                email=Email,

                first_name=First_Name,

                is_staff=admin

            )

            user.save()

            return redirect('login')

    else:

        messages.error(request, 'Passwords do not match.')

return render(request, 'register.html')
```

```
    return render(request, 'register.html')

def login_view(request):

    if request.method == "POST":

        username = request.POST.get('username')

        password = request.POST.get('password')

        if User.objects.filter(username=username).exists():

            user=User.objects.get(username=username)

            if user.check_password(password):

                user = authenticate(username=username,password=password)

                if user is not None:

                    login(request,user)

                    messages.success(request,'login successfull')

                    return redirect('/')

                else:

                    messages.error(request,'please check the Password Properly')

                    return redirect('login')

            else:

                messages.error(request,"please check the Password Properly")

                return redirect('login')

        else:

            messages.error(request,"username doesn't exist")

            return redirect('login')
```

```
        return render(request,'login.html')

def logout_view(request):

    logout(request)

    return redirect('login')


import os


# Ensure 'static/images' directory exists

os.makedirs(os.path.join('static', 'images'), exist_ok=True)

chart_path = os.path.join('static', 'images', 'performance_chart.png')


from django.shortcuts import render

from django.core.files.storage import default_storage

import pandas as pd

import numpy as np

import os


from sklearn.preprocessing import LabelEncoder

from tensorflow.keras.models import load_model


# Global variables

X = y = df = None

le = LabelEncoder()
```

```
labels = ["failure","normal"] # Based on earlier context
```

```
# ----- UPLOAD FUNCTION -----
```

```
def Upload_data(request):
```

```
    load = True
```

```
    global df, x, y
```

```
    if request.method == 'POST' and request.FILES.get('file'):
```

```
        uploaded_file = request.FILES['file']
```

```
        file_path = default_storage.save(uploaded_file.name, uploaded_file)
```

```
        # Read uploaded file
```

```
        df = pd.read_csv(default_storage.path(file_path), low_memory=False)
```

```
        df.drop(["pump_status"], axis=1, inplace=True)
```

```
        # Drop unwanted columns
```

```
        # Delete file after reading
```

```
        default_storage.delete(file_path)
```

```
        # Preview uploaded data
```

```

outdata = df.head(100)

        return render(request, 'prediction.html', {'predict':
outdata.to_html(classes='table table-striped')})

return render(request, 'prediction.html', {'upload': load})

# ----- PREPROCESS FUNCTION -----
# ----- PREPROCESS FUNCTION -----

global x_train, x_test, y_train, y_test

def preprocess(request):

    global df, x, y, le,x_train,x_test,y_train,y_test

    # Label encode categorical columns

    for col in df.columns:

        if df[col].dtype == 'object' or df[col].dtype == 'bool':

            df[col] = le.fit_transform(df[col].astype(str))

    # Save processed DataFrame

    df_encoded =df.copy()

    df_encoded['maintenance_required'] = y

```

```

# Save to CSV

df_encoded.to_csv('agri_predictive_maintenance_encoded.csv',
index=False)

# Preview first 10 rows

preview_df = df_encoded.head(10)

x= df.drop('maintenance_required', axis=1)

y = df['maintenance_required']

from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_s
tate=42)

return render(request, 'prediction.html', {

    'message': '  Preprocessing completed and
credit_card_fraud_encoded.csv saved.',

    'preview': preview_df.to_html(classes='table table-bordered table-sm')

})

from django.shortcuts import render

import numpy as np

import pandas as pd

import random

```

```
import os

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import cross_val_score, train_test_split

from sklearn.metrics import accuracy_score, classification_report


# Global variables to access processed features

#X_selected = y_selected = None


precision = []

recall = []

fscore = []

accuracy = []

import threading

def calculateMetrics(algorithm, predict, testY):

    # Calculate performance metrics

    testY = testY.astype('int64')

    predict = predict.astype('int64')

    p = precision_score(testY, predict, average='macro') * 100
```

```
r = recall_score(testY, predict, average='macro') * 100

f = f1_score(testY, predict, average='macro') * 100

a = accuracy_score(testY, predict) * 100

accuracy.append(a)

precision.append(p)

recall.append(r)

fscore.append(f)


# Print metrics

print(f'{algorithm} Accuracy   : {a}')

print(f'{algorithm} Precision : {p}')

print(f'{algorithm} Recall    : {r}')

print(f'{algorithm} FSCORE    : {f}')


# Classification report

report = classification_report(testY, predict, target_names=Label)

print(f'\n{algorithm} classification report\n{report}')


# Confusion matrix

conf_matrix = confusion_matrix(testY, predict)


# Create the plot in the main thread to avoid potential issues
```

```

def show_plot():

    plt.figure(figsize=(5, 5))

    ax = sns.heatmap(conf_matrix, xticklabels=Label, yticklabels=Label,
annot=True, cmap="Spectral", fmt="g")

    ax.set_ylim([0, len(Label)])

    plt.title(f'{algorithm} Confusion matrix')

    plt.ylabel('True class')

    plt.xlabel('Predicted class')

    plt.show()

    plt.close()

# Execute the plot display in the main thread
if threading.current_thread().name == "MainThread":

    show_plot()

else:

    # Use this if needed for threading environments

    threading.Thread(target=show_plot).start()

```

```

def train_lstm_view(request):

    import os

    import numpy as np

    from tensorflow.keras.models import Sequential, load_model

```

```

from tensorflow.keras.layers import LSTM, Dense

from tensorflow.keras.callbacks import EarlyStopping

# Ensure the model directory exists

os.makedirs('model', exist_ok=True)

model_path = 'model/LSTM_Model.h5'


# Reshape x_train and x_test for LSTM

x_train_lstm = x_train.values.reshape((x_train.shape[0], 1,
x_train.shape[1]))

x_test_lstm = x_test.values.reshape((x_test.shape[0], 1, x_test.shape[1]))


# Load or train the model

if os.path.exists(model_path):

    lstm_model = load_model(model_path)

    print("✅ LSTM model loaded successfully.")

else:

    print("❌ Model not found. Training LSTM model...")

    timesteps = x_train_lstm.shape[1]

    features = x_train_lstm.shape[2]

    lstm_model = Sequential()

```

```
lstm_model.add(LSTM(64, input_shape=(timesteps, features),
return_sequences=False))
```

```
lstm_model.add(Dense(32, activation='relu'))
```

```
lstm_model.add(Dense(1, activation='sigmoid')) # For binary
classification
```

```
lstm_model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

```
# Early stopping
```

```
early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
```

```
lstm_model.fit(
    x_train_lstm, y_train,
    epochs=50, batch_size=64,
    validation_split=0.2,
    callbacks=[early_stop]
)
```

```
lstm_model.save(model_path)
```

```
print("✅ LSTM model trained and saved.")
```

Prediction

y_pred = (lstm_model.predict(x_test_lstm) > 0.5).astype(int)

Evaluation using shared metrics function

calculateMetrics("LSTM_Model", y_pred, y_test)

return render(request, 'prediction.html', {

'algorithm': 'LSTM_Model',

'accuracy': f"{accuracy[-1]:.2f}%",

'precision': f"{precision[-1]:.2f}%",

'recall': f"{recall[-1]:.2f}%",

'fscore': f"{fscore[-1]:.2f}%"

})

def train_bilstm_view(request):

import os

import numpy as np

from tensorflow.keras.models import Sequential, load_model

from tensorflow.keras.layers import LSTM, Dense, Bidirectional

from tensorflow.keras.callbacks import EarlyStopping

```

# Ensure the model directory exists

os.makedirs('model', exist_ok=True)

model_path = 'model/BiLSTM_Model.h5'


# Reshape x_train and x_test for BiLSTM

x_train_bilstm = x_train.values.reshape((x_train.shape[0], 1,
x_train.shape[1]))

x_test_bilstm = x_test.values.reshape((x_test.shape[0], 1, x_test.shape[1]))


# Load or train the model

if os.path.exists(model_path):

    bilstm_model = load_model(model_path)

    print("✅ BiLSTM model loaded successfully.")

else:

    print("❌ Model not found. Training BiLSTM model...")

    timesteps = x_train_bilstm.shape[1]

    features = x_train_bilstm.shape[2]

    bilstm_model = Sequential()

    bilstm_model.add(Bidirectional(LSTM(64, return_sequences=False),
input_shape=(timesteps, features)))

    bilstm_model.add(Dense(32, activation='relu'))

```

```
bilstm_model.add(Dense(1, activation='sigmoid')) # For binary  
classification
```

```
bilstm_model.compile(loss='binary_crossentropy', optimizer='adam',  
metrics=['accuracy'])
```

```
# Early stopping
```

```
early_stop = EarlyStopping(monitor='val_loss', patience=5,  
restore_best_weights=True)
```

```
bilstm_model.fit(  
    x_train_bilstm, y_train,  
    epochs=50, batch_size=64,  
    validation_split=0.2,  
    callbacks=[early_stop]  
)
```

```
bilstm_model.save(model_path)
```

```
print("👍 BiLSTM model trained and saved.")
```

```
# Prediction
```

```
y_pred = (bilstm_model.predict(x_test_bilstm) > 0.5).astype(int)
```

```
# Evaluation using shared metrics function
```

```
calculateMetrics("BiLSTM_Model", y_pred, y_test)
```

```
return render(request, 'prediction.html', {
```

```
    'algorithm': 'BiLSTM_Model',
```

```
    'accuracy': f"{accuracy[-1]:.2f}%",
```

```
    'precision': f"{precision[-1]:.2f}%",
```

```
    'recall': f"{recall[-1]:.2f}%",
```

```
    'fscore': f"{fscore[-1]:.2f}%"
```

```
})
```

```
def train_ann_view(request):
```

```
    import os
```

```
    import numpy as np
```

```
    from tensorflow.keras.models import Sequential, load_model
```

```
    from tensorflow.keras.layers import Dense
```

```
    from tensorflow.keras.callbacks import EarlyStopping
```

```
# Ensure the model directory exists
```

```
os.makedirs('model', exist_ok=True)
```

```
model_path = 'model/ANN_Model.h5'
```

```
# ANN uses original features (no reshaping needed)

x_train_ann = x_train.values

x_test_ann = x_test.values


# Load or train the model

if os.path.exists(model_path):

    ann_model = load_model(model_path)

    print("✅ ANN model loaded successfully.")

else:

    print("❌ Model not found. Training ANN model...")

    features = x_train_ann.shape[1]

    ann_model = Sequential()

    ann_model.add(Dense(128, activation='relu', input_dim=features))

    ann_model.add(Dense(64, activation='relu'))

    ann_model.add(Dense(32, activation='relu'))

    ann_model.add(Dense(1, activation='sigmoid')) # For binary
classification

    ann_model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

```
# Early stopping  
  
    early_stop = EarlyStopping(monitor='val_loss', patience=5,  
restore_best_weights=True)
```

```
ann_model.fit(  
  
    x_train_ann, y_train,  
  
    epochs=50, batch_size=64,  
  
    validation_split=0.2,  
  
    callbacks=[early_stop]  
  
)
```

```
ann_model.save(model_path)  
  
print("✅ ANN model trained and saved.")
```

```
# Prediction
```

```
y_pred = (ann_model.predict(x_test_ann) > 0.5).astype(int)
```

```
# Evaluation using shared metrics function
```

```
calculateMetrics("ANN_Model", y_pred, y_test)
```

```
return render(request, 'prediction.html', {
```

```
'algorithm': 'ANN_Model',  
'accuracy': f'{accuracy[-1]:.2f}%',  
'precision': f'{precision[-1]:.2f}%',  
'recall': f'{recall[-1]:.2f}%',  
'fscore': f'{fscore[-1]:.2f}%'  
)
```

```
def train_lgbm_view(request):  
  
import os  
  
import numpy as np  
  
import lightgbm as lgb  
  
from joblib import dump, load  
  
  
# Ensure the model directory exists  
  
os.makedirs('model', exist_ok=True)  
  
model_path = 'model/LGBM_Model.pkl'  
  
  
  
  
# Convert x_train and x_test to numpy arrays  
  
x_train_lgb = x_train.values  
  
x_test_lgb = x_test.values  
  
y_train_lgb = y_train.values  
  
y_test_lgb = y_test.values
```

Load or train the model

if os.path.exists(model_path):

lgbm_model = load(model_path)

print(" ✓ LGBM model loaded successfully.")

else:

print(" ✗ Model not found. Training LGBM model...")

lgbm_model = lgb.LGBMClassifier(

n_estimators=500,

learning_rate=0.05,

max_depth=-1,

random_state=42

)

Train the model

lgbm_model.fit(

x_train_lgb, y_train_lgb,

eval_set=[(x_test_lgb, y_test_lgb)],

eval_metric='binary_logloss',

callbacks=[lgb.early_stopping(stopping_rounds=20, verbose=False)]

)

Save the model

dump(lgbm_model, model_path)

print("✅ LGBM model trained and saved.")

Prediction

y_pred = lgbm_model.predict(x_test_lgb)

Evaluation using shared metrics function

calculateMetrics("LGBM_Model", y_test_lgb, y_pred)

return render(request, 'prediction.html', {

'algorithm': 'LGBM_Model',

'accuracy': f"{accuracy[-1]:.2f}%",

'precision': f"{precision[-1]:.2f}%",

'recall': f"{recall[-1]:.2f}%",

'fscore': f"{fscore[-1]:.2f}%"

})

import os

import pandas as pd

```
import matplotlib.pyplot as plt
```

```
from django.shortcuts import render
```

```
# Global metrics (you should ensure these are defined somewhere before this  
view is called)
```

```
accuracy = []
```

```
precision = []
```

```
recall = []
```

```
fscore = []
```

```
def performance_summary_view(request):
```

```
    global accuracy, precision, recall, fscore
```

```
    algorithm_names = ["LSTM", "BI_LSTM", "ANN", "LGBM"]
```

```
    if len(accuracy) < len(algorithm_names):
```

```
        return render(request, 'prediction.html', {'error': ' ✖ Not all models  
have been evaluated yet.'})
```

```
        columns = ["Algorithm Name", "Precision", "Recall", "F1 Score",  
"Accuracy"]
```

```
        values = []
```

```
for i in range(len(algorithm_names)):
```

```
    values.append([
```

```
        algorithm_names[i],
```

```
        round(precision[i], 2),
```

```
        round(recall[i], 2),
```

```
        round(fscore[i], 2),
```

```
        round(accuracy[i], 2)
```

```
    ])
```

```
temp = pd.DataFrame(values, columns=columns)
```

```
# Save performance chart to static/images/
```

```
static_images_dir = os.path.join('static', 'images')
```

```
os.makedirs(static_images_dir, exist_ok=True) # Ensure directory exists
```

```
chart_path = os.path.join(static_images_dir, 'performance_chart.png')
```

```
# Plot the chart
```

```
fig, axs = plt.subplots(2, 2, figsize=(10, 10))
```

```
fig.suptitle("Performance Metrics of Different Models", fontsize=16)
```

```
    axs[0, 0].bar(temp["Algorithm Name"], temp["Precision"],
color='skyblue')
```

```
    axs[0, 0].set_title("Precision")
```

```
    axs[0, 0].tick_params(axis='x', rotation=15)
```

```
    axs[0, 1].bar(temp["Algorithm Name"], temp["Recall"], color='orange')
```

```
    axs[0, 1].set_title("Recall")
```

```
    axs[0, 1].tick_params(axis='x', rotation=15)
```

```
    axs[1, 0].bar(temp["Algorithm Name"], temp["F1 Score"], color='green')
```

```
    axs[1, 0].set_title("F1 Score")
```

```
    axs[1, 0].tick_params(axis='x', rotation=15)
```

```
    axs[1, 1].bar(temp["Algorithm Name"], temp["Accuracy"],
color='purple')
```

```
    axs[1, 1].set_title("Accuracy")
```

```
    axs[1, 1].tick_params(axis='x', rotation=15)
```

```
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
```

```
plt.savefig(chart_path)
```

```
plt.close()
```

```
return render(request, 'prediction.html', {  
    'table': temp.to_html(classes='table table-bordered table-hover',  
index=False),  
    'chart_url': '/static/images/performance_chart.png'  
})
```

```
def predict_view(request):
```

```
    if request.method == 'POST' and request.FILES.get('file'):
```

```
        import pandas as pd
```

```
        import numpy as np
```

```
        import joblib
```

```
        from django.core.files.storage import default_storage
```

```
        labels = ["failure", "normal"]
```

```
        # Read uploaded CSV file
```

```
        uploaded_file = request.FILES['file']
```

```
        file_path = default_storage.save(uploaded_file.name, uploaded_file)
```

```
        test = pd.read_csv(default_storage.path(file_path))
```

```
        default_storage.delete(file_path)
```

```
        # Ensure 16 features
```

```
if test.shape[1] < 8:

    for i in range(8- test.shape[1]):

        test[f'dummy_{i}'] = 0

elif test.shape[1] > 8:

    test = test.iloc[:, :8]


# Convert to array (2D)

test_array = test.values


# Load stacked model

model = joblib.load('model/LGBM_Model.pkl')


# Predict

predictions = model.predict(test_array)


# Map predictions to labels

test['Predicted_Label'] = [labels[int(p)] for p in predictions]


return render(request, 'prediction.html', {

    'predict': test.head(100).to_html(classes='table table-bordered',
index=False)

})
```

```
return render(request, 'prediction.html', {'test': True})
```

MODULES

Home & About Module

- Displays the homepage and project information.
- Acts as the entry point where users can navigate to login, register, or view project details.

User Management Module

- Handles **registration, login, and logout** functionalities.
- Supports roles: **Admin** (manages data and models) and **User** (can test predictions).
- Ensures authentication and secure access to system features.

Dataset Upload Module

- Allows the admin to upload the agricultural predictive maintenance dataset (agri_predictive_maintenance.csv).
- Uploaded data is stored temporarily, pre-processed, and displayed for preview.

Preprocessing Module

- Encodes categorical variables and splits the dataset into training and testing sets.
- Generates a cleaned dataset for consistent model training.

Model Training Module

- Implements four algorithms: **LSTM, BiLSTM, ANN, and LightGBM**.
- Each model is trained, saved, and evaluated using accuracy, precision, recall, and F1-score.
- Performance metrics and confusion matrices are generated.

Performance Analysis Module

- Compares the performance of all trained models.
- Displays results in both tabular format and graphical bar charts (Precision, Recall, F1-score, Accuracy).

Prediction Module

- Accepts new test data files.
- Loads the trained LightGBM model (or other models) to predict whether maintenance is required.
- Displays results with predicted labels (failure or normal).

.

Dataset description:

Dataset Name: agri_predictive_maintenance.csv

Number of Rows: 50,000

Number of Columns: 10 (9 features + 1 target)

Target Variable: maintenance_required (0 = No maintenance required, 1 = Maintenance required)

Feature Descriptions:

temperature – Ambient temperature in the facility (°C). Normally distributed around 25°C with $\pm 5^\circ\text{C}$ variance.

humidity – Relative humidity in the facility (%). Uniformly distributed between 30% and 90%.

soil_moisture – Soil moisture percentage. Uniformly distributed between 10% and 50%.

vibration_sensor_1 – Vibration level from the first machinery sensor. Normally distributed around 0.5 ± 0.1 . High values indicate potential faults.

vibration_sensor_2 – Vibration level from the second machinery sensor. Normally distributed around 0.5 ± 0.1 .

motor_current – Electric current drawn by machinery motors (Amps). Normally distributed around 5 ± 1 . High current may indicate abnormal operation.

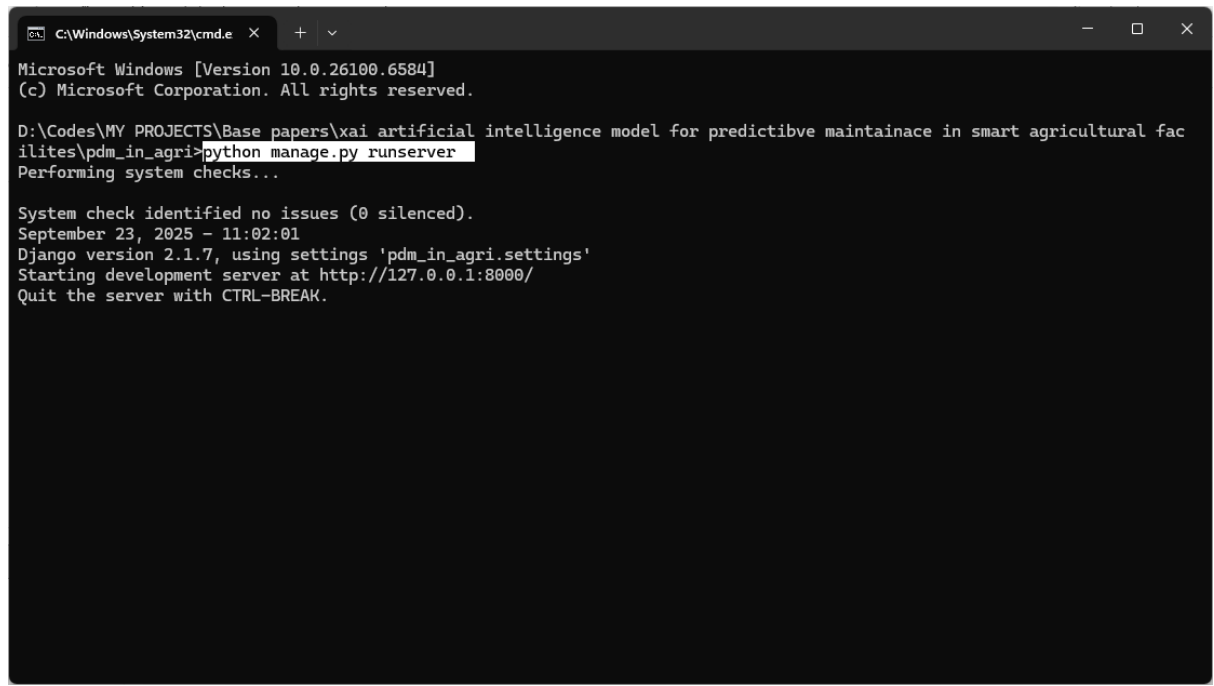
pump_status – Operational status of the pump (0 = normal, 1 = failure). Simulated with 5% failure probability.

last_maintenance_days – Days since the last maintenance event. Integer values from 0 to 365. High values may trigger maintenance.

error_code – Machinery error code (0 = None, 1 = Minor, 2 = Critical). Higher codes contribute to maintenance decisions.

maintenance_required – Target variable (0 = No, 1 = Yes). Determined based on a combination of pump status, vibration, motor current, last maintenance days, and error code.

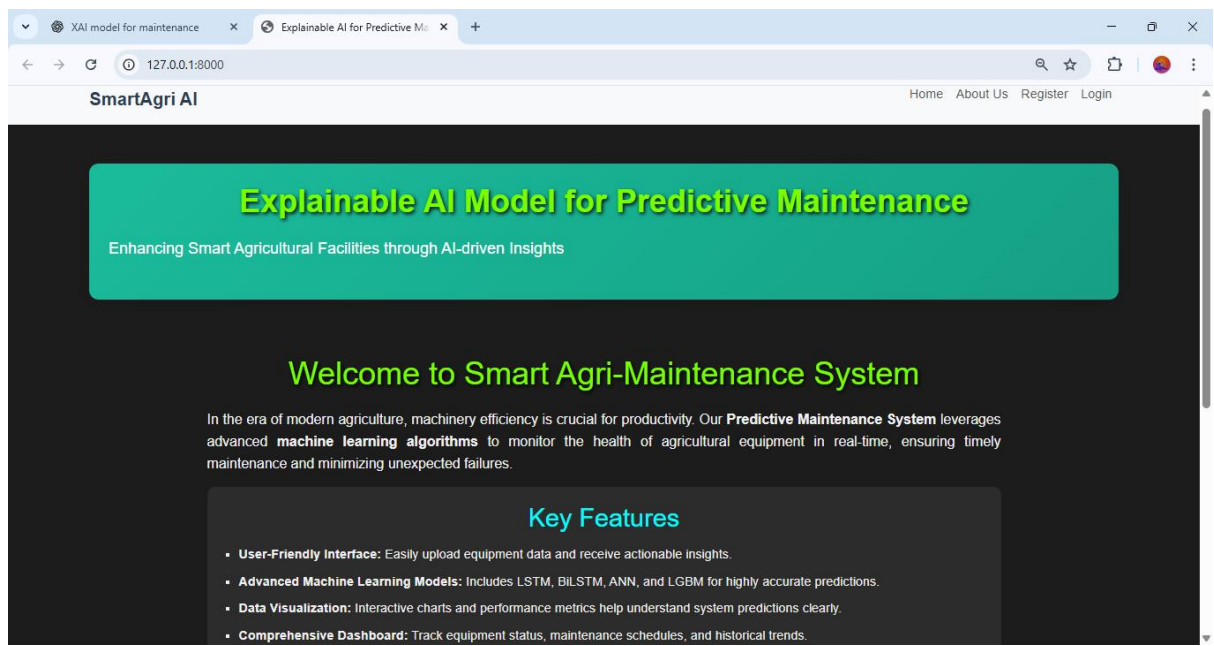
SCREEN SHOTS

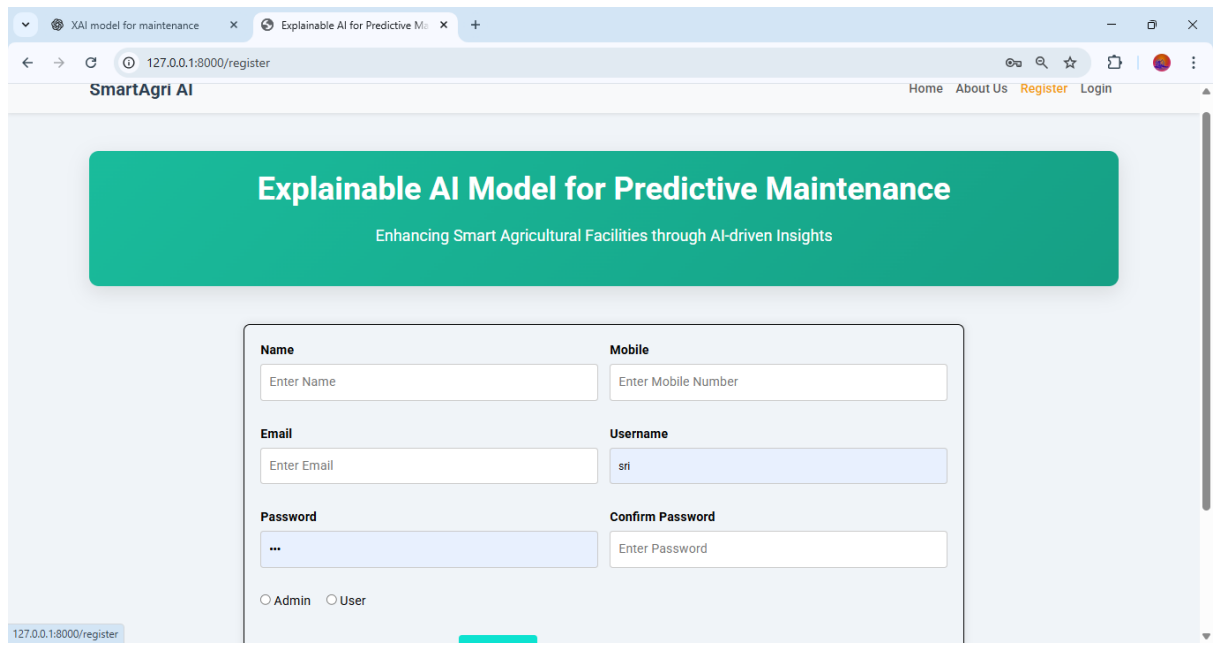
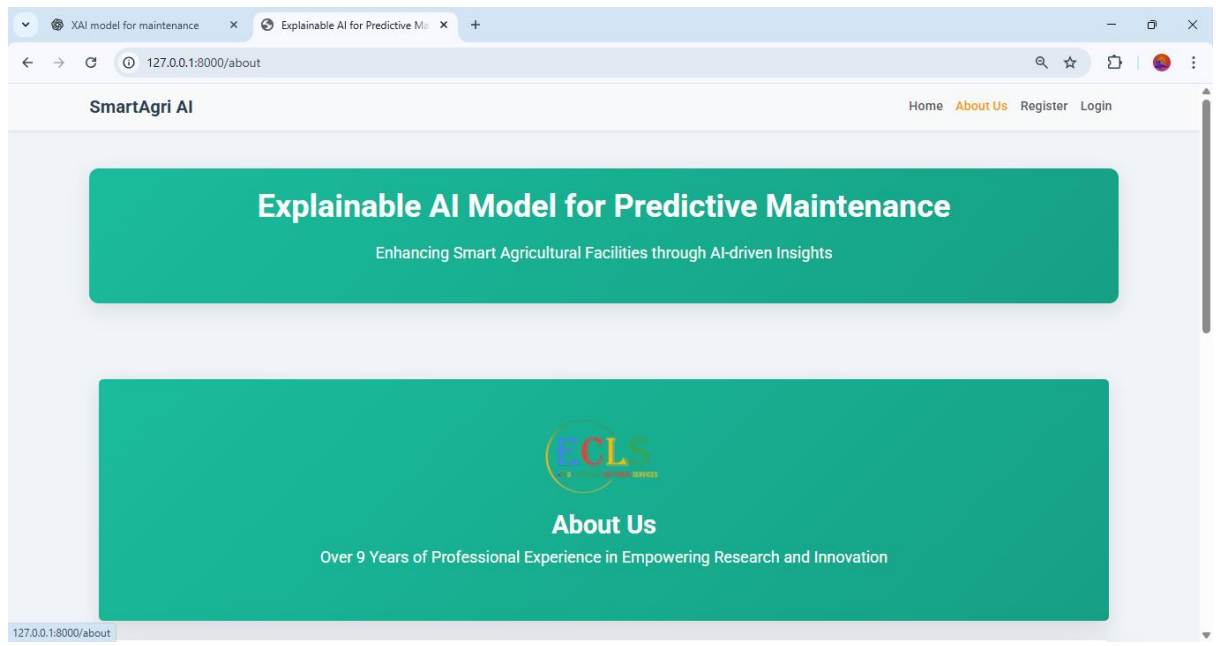


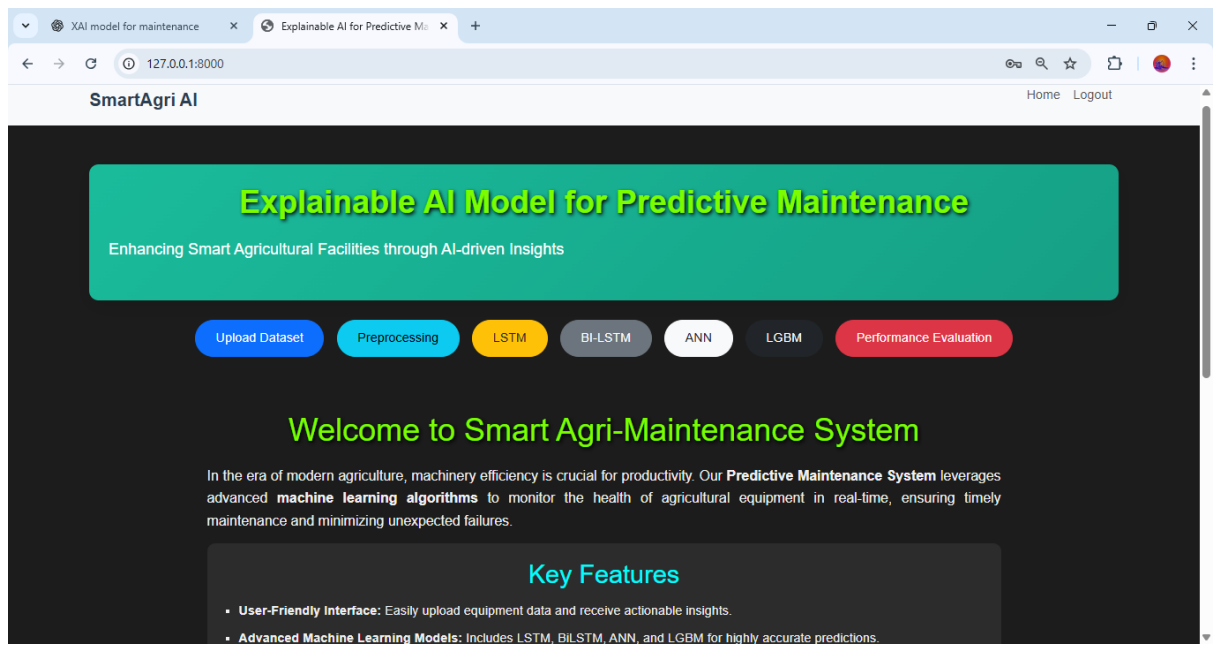
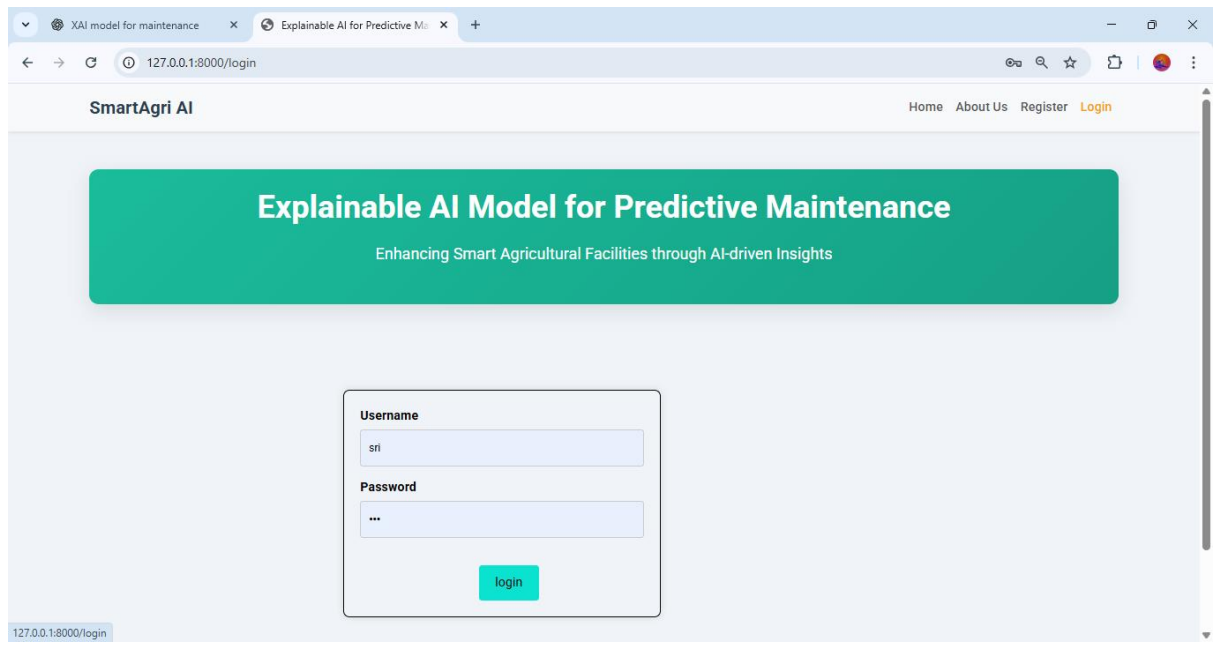
```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

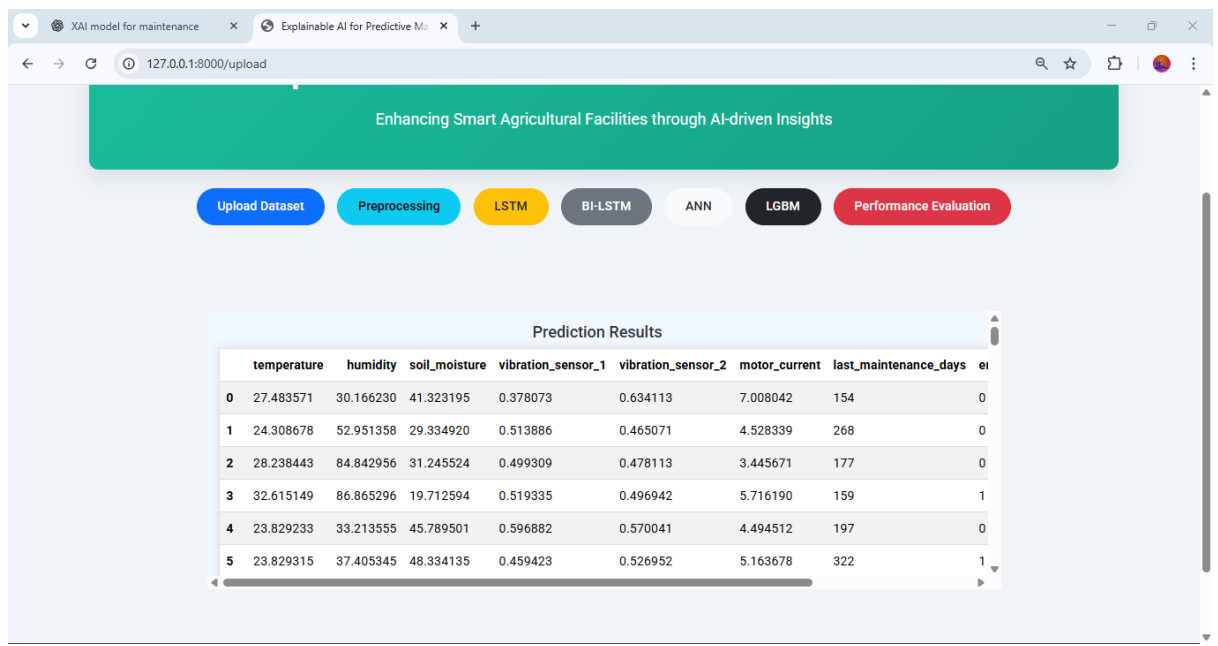
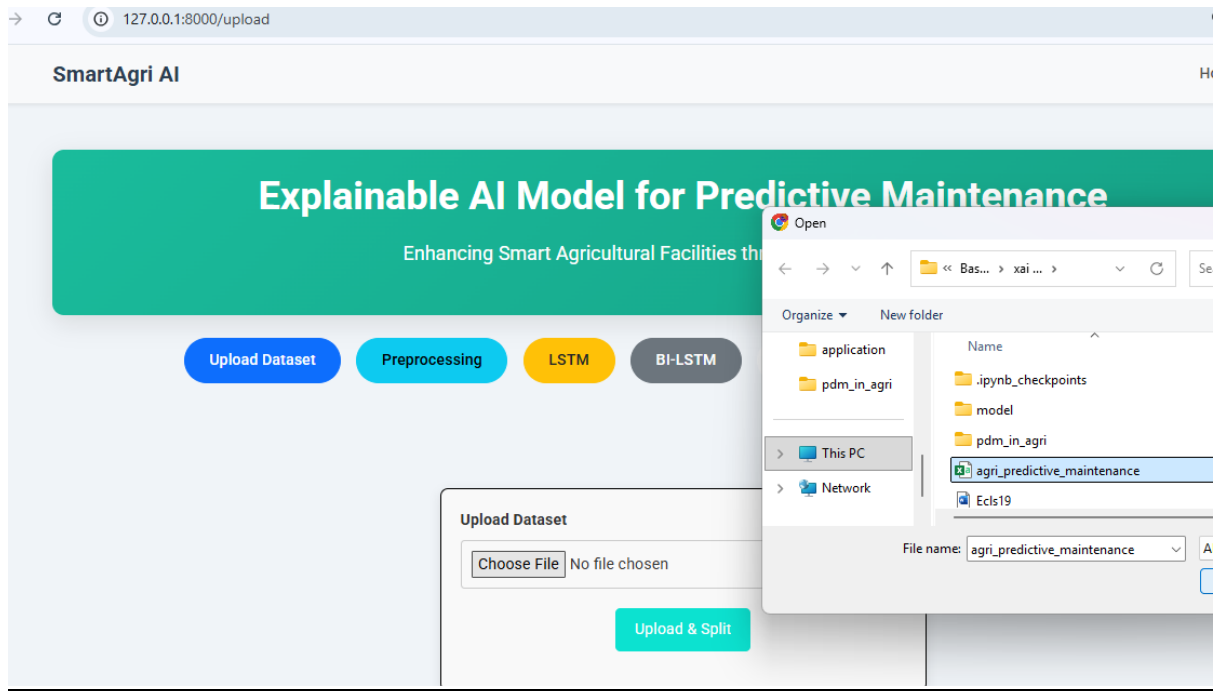
D:\Codes\MY PROJECTS\Base papers\xai artificial intelligence model for predictibve maintainace in smart agricultural fac
ilites\pdm_in_agri>python manage.py runserver
Performing system checks...

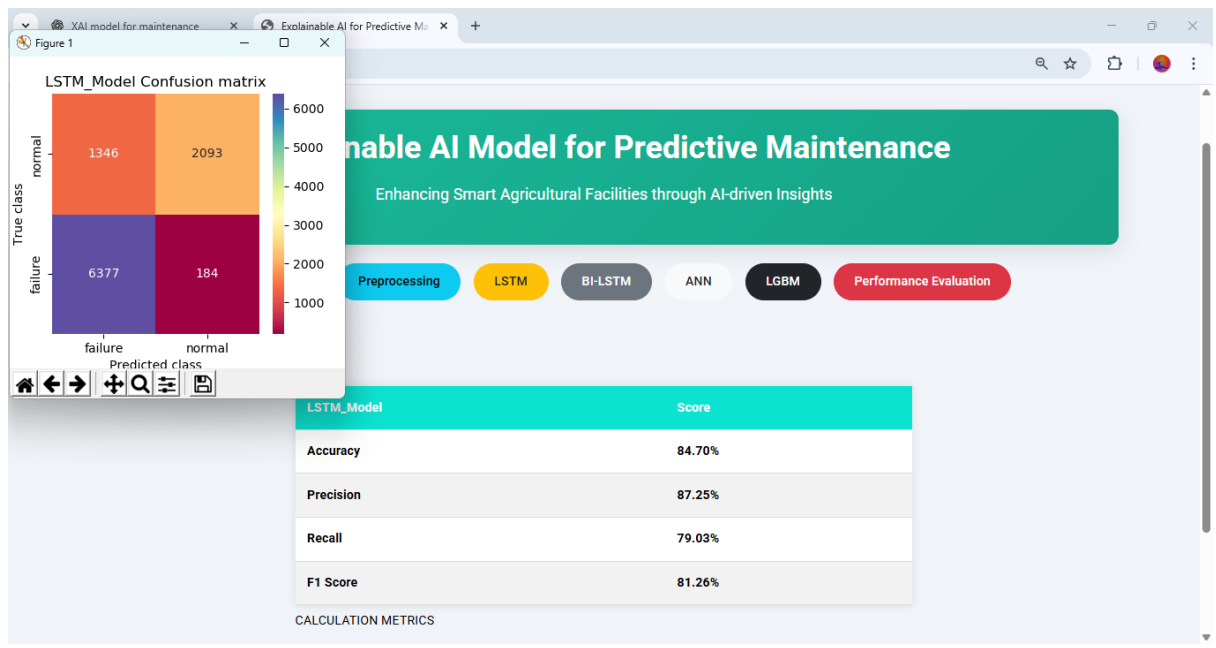
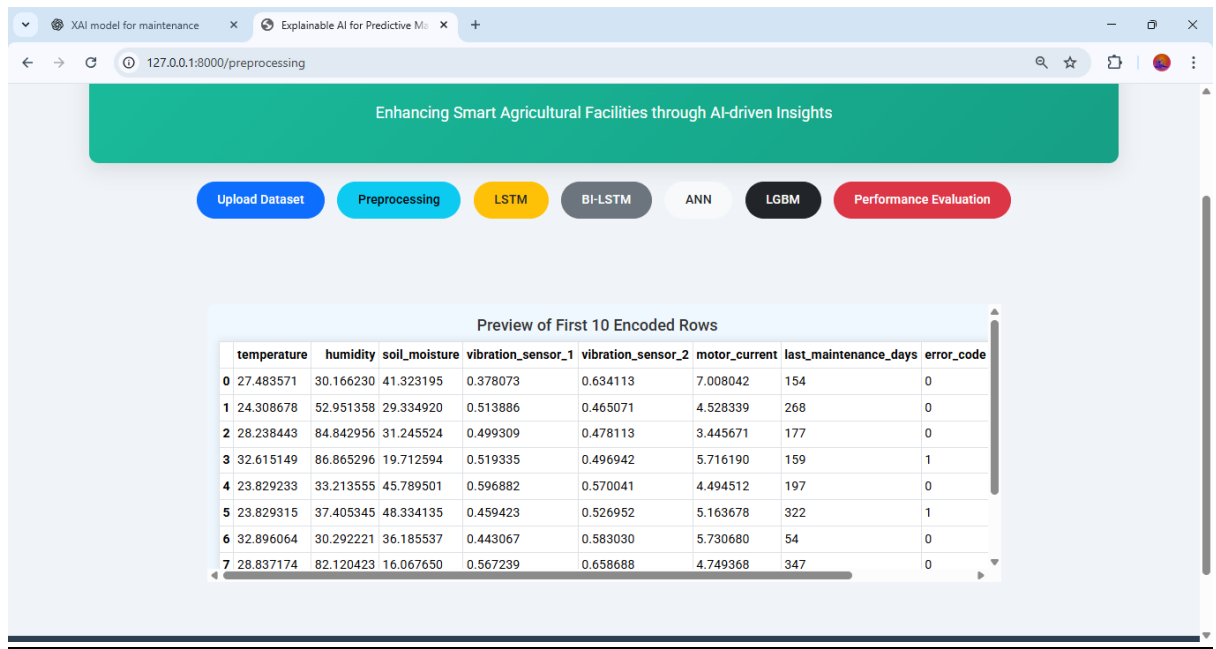
System check identified no issues (0 silenced).
September 23, 2025 - 11:02:01
Django version 2.1.7, using settings 'pdm_in_agri.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

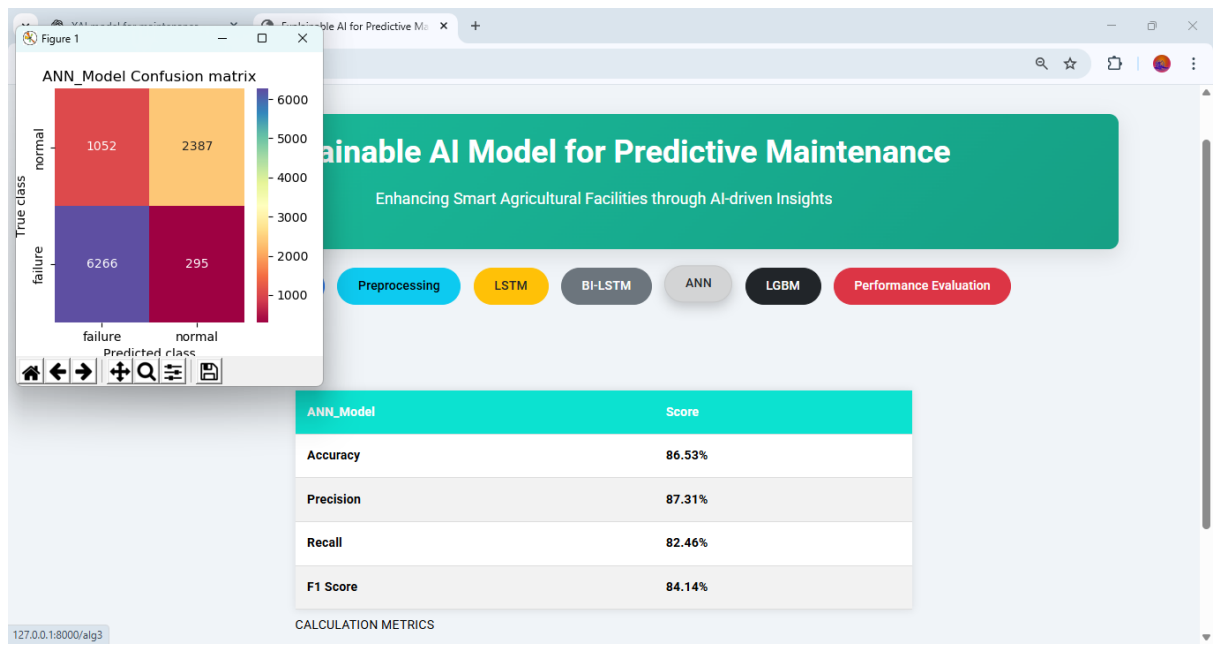
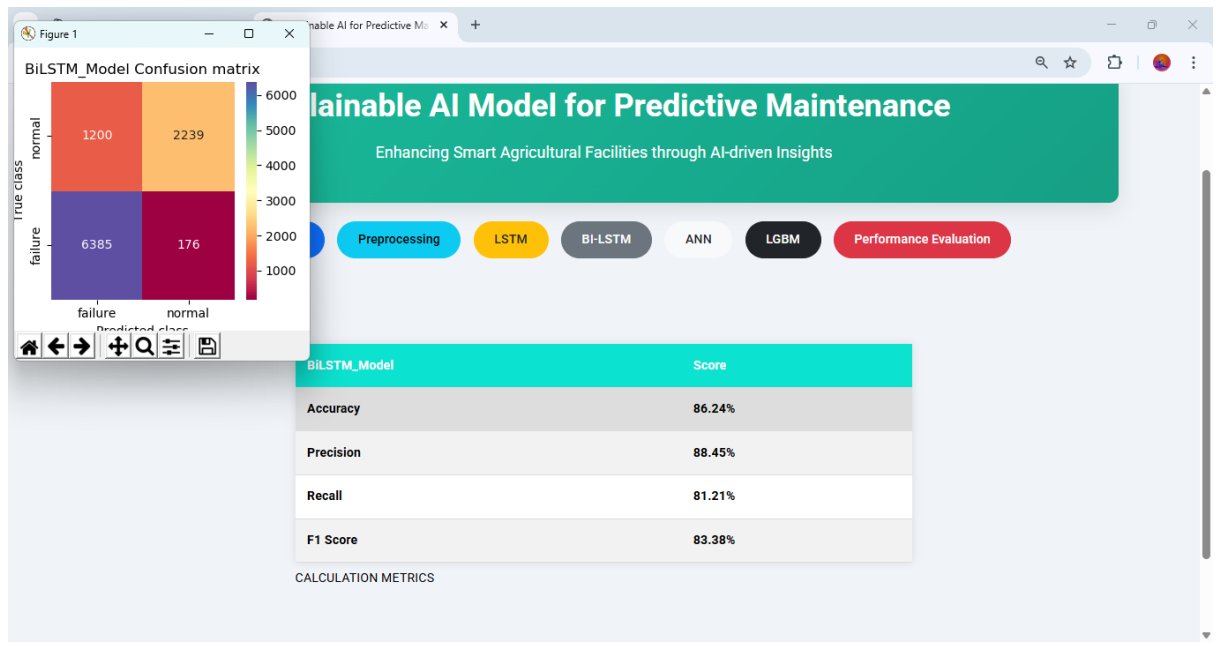


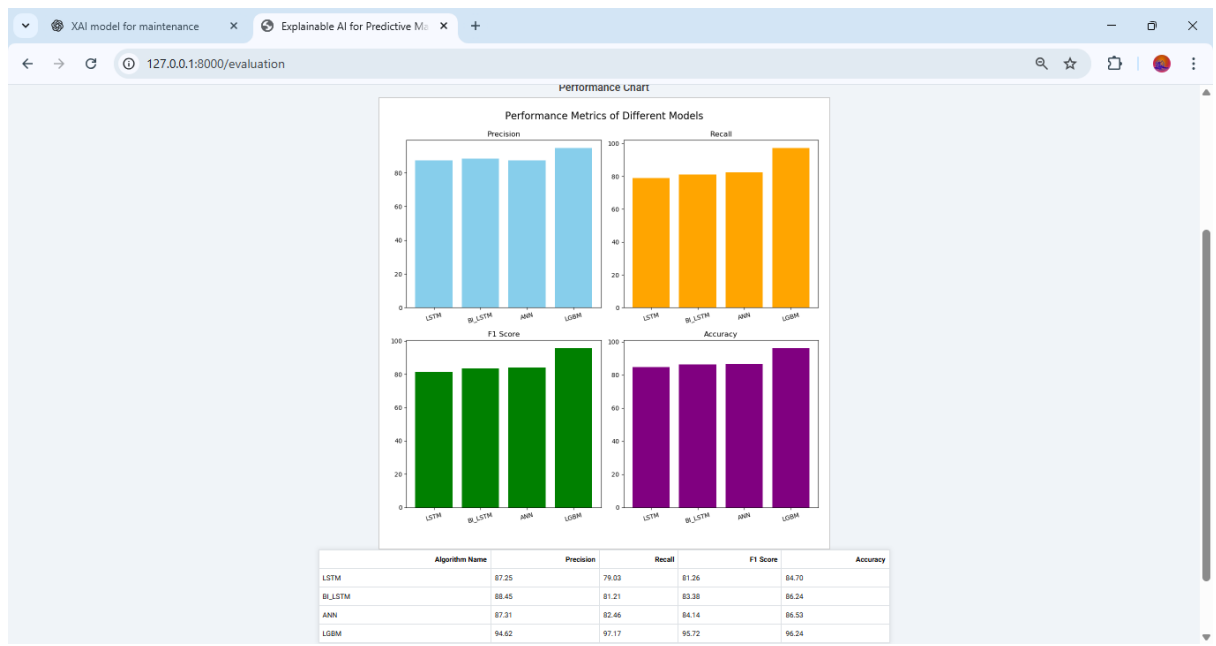
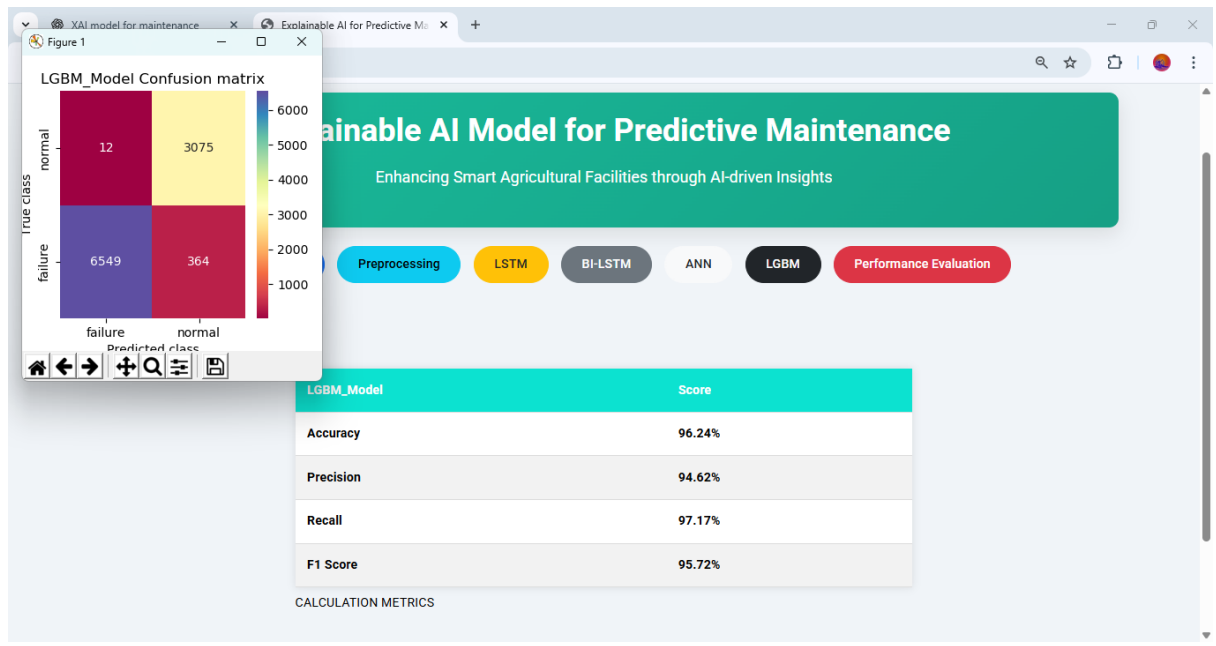


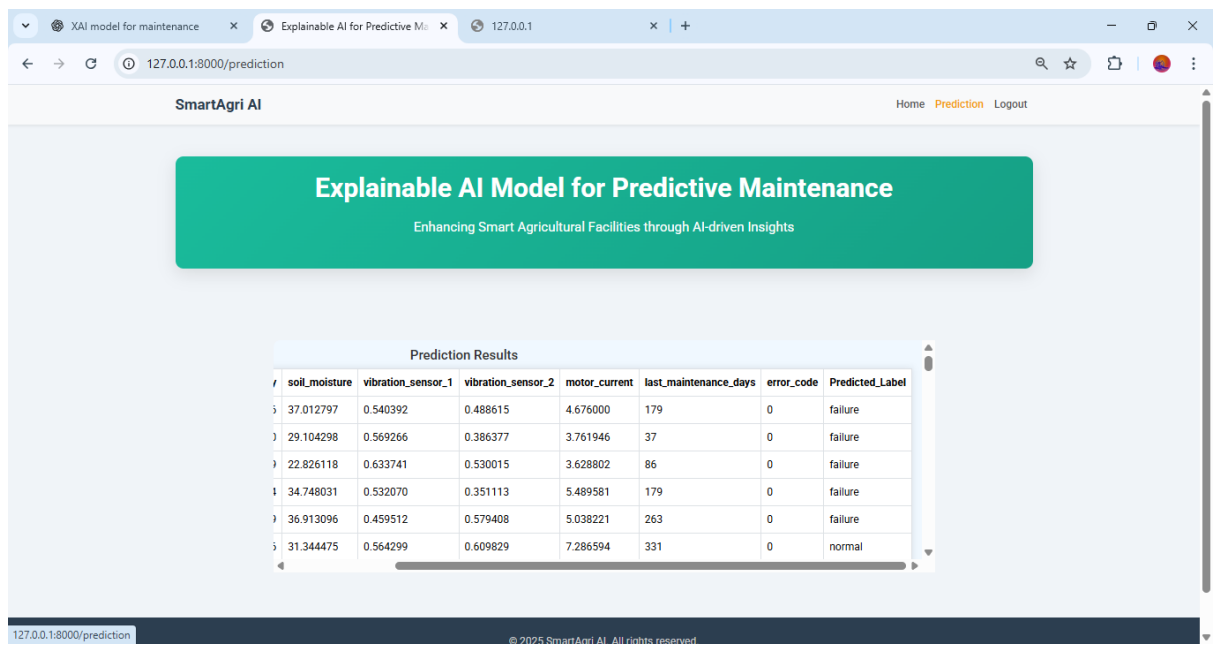
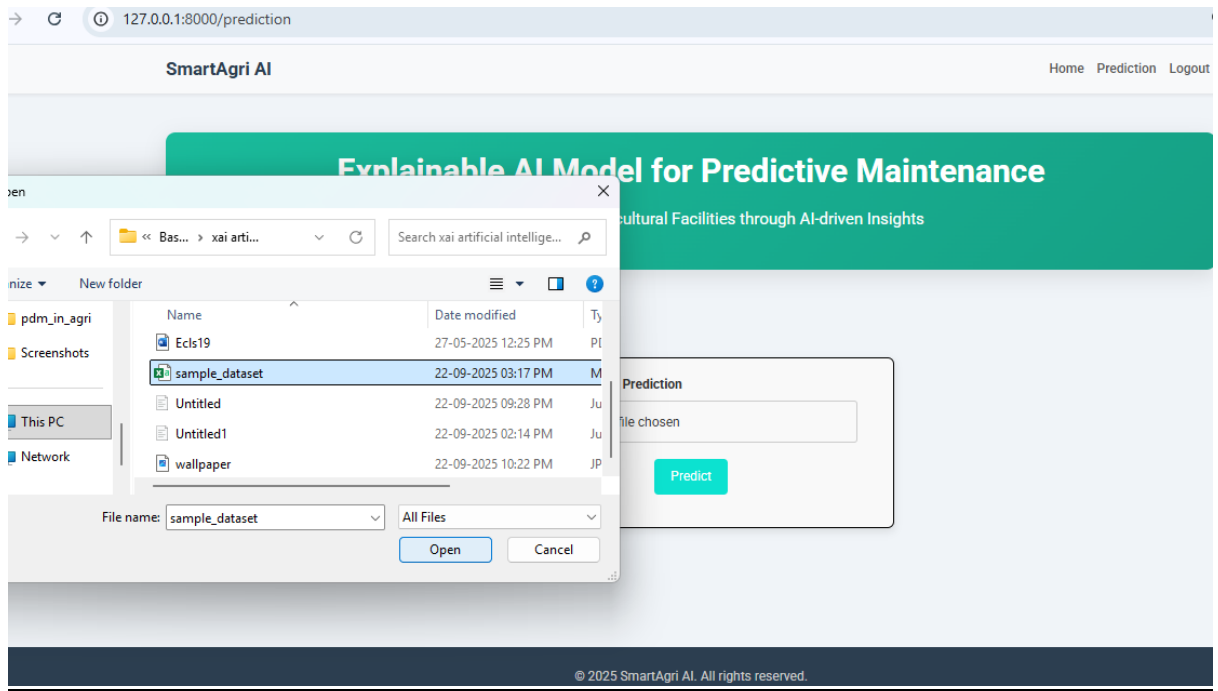












CONCLUSION

This project demonstrates the development of an **Explainable Artificial Intelligence (XAI)–based predictive maintenance system for smart agricultural facilities**, addressing the limitations of traditional corrective and preventive methods. By leveraging real-time sensor data and advanced machine learning models such as LSTM, BiLSTM, ANN, and LightGBM, the system can accurately predict potential equipment failures before they occur. The integration of explainability ensures that predictions are not only accurate but also interpretable, fostering greater trust among farmers and facility managers. Implemented through a Django-based platform, the system offers modules for dataset management, preprocessing, model training, performance analysis, and real-time predictions, making it practical and user-friendly. Ultimately, the proposed solution reduces downtime, lowers maintenance costs, optimizes resource utilization, and supports sustainable agricultural practices. This work highlights the transformative potential of AI in agriculture, paving the way for more reliable, transparent, and intelligent farm management systems.

FUTURE SCOPE OF THE PROJECT

The proposed system provides a solid foundation for predictive maintenance in smart agricultural facilities; however, there is significant potential for further enhancement. One promising direction is the **integration of IoT-enabled real-time monitoring systems**, where sensor data can be continuously streamed from agricultural equipment to the predictive model. This would enable immediate fault detection and proactive interventions, further reducing downtime.

Another future improvement involves **cloud-based deployment and edge computing**. By hosting the system on the cloud, large-scale datasets from multiple farms can be processed collaboratively, enabling model updates in real time. Edge computing, on the other hand, would allow predictive insights to be generated directly on-site, even in remote agricultural locations with limited internet connectivity.

Additionally, the system can be extended with **reinforcement learning** and **adaptive algorithms** that dynamically learn from new equipment behaviors, improving accuracy over time. The inclusion of **multimodal data sources**, such as drone imagery, weather data, and satellite observations, could provide richer insights into equipment performance and environmental interactions.

Finally, incorporating a **mobile application interface** would allow farmers to receive real-time alerts, maintenance schedules, and explainable insights directly on their devices. Such advancements will transform the system into a comprehensive decision-support tool that is scalable, adaptive, and accessible, ultimately contributing to smarter, more sustainable agricultural practices.

REFERENCES

1. Kisten, M., Ezugwu, A. E.-S., & Olusanya, M. O. (2024). Explainable Artificial Intelligence Model for Predictive Maintenance in Smart Agricultural Facilities. *IEEE Access*, 12, 24348-24367. [ResearchGate](#)
2. Molęda, M. (2023). From Corrective to Predictive Maintenance — A Review. *PMC*. [PMC](#)
3. Shaif, A. (2021). Predictive Maintenance in Smart Agriculture Using Machine Learning. (Master's thesis) [Diva Portal](#)
4. Hrast Essenfelder, A., Toreti, A., & Seguni, L. (2025). Expert-driven explainable artificial intelligence models can detect multiple climate hazards relevant for agriculture. *Communications Earth & Environment*, 6, 207. [Nature](#)
5. Abdallah, M., Lee, W. J., Raghunathan, N., Mousoulis, C., Sutherland, J. W., & Bagchi, S. (2021). Anomaly Detection through Transfer Learning in Agriculture and Manufacturing IoT Systems. arXiv preprint. [arXiv](#)
6. Ngo, Q. H., Kechadi, T., & Le-Khac, N.-A. (2022). OAK4XAI: Model towards Out-Of-Box eXplainable Artificial Intelligence for Digital Agriculture. arXiv preprint. [arXiv](#)
7. Turgut, O., Kok, I., & Ozdemir, S. (2024). AgroXAI: Explainable AI-Driven Crop Recommendation System for Agriculture 4.0. arXiv. [arXiv](#)
8. Aijaz, N., et al. (2025). Artificial intelligence in agriculture: Advancing crop management. *ScienceDirect*. [ScienceDirect](#)
9. "Predictive maintenance techniques for enhancing the lifespan of agri-machinery." (2025). ResearchGate publication. [ResearchGate](#)
10. "Redictive Maintenance of Smart Agriculture Using Explainable Ai (XAI)." (2025). IJSAT. [ijsat.org](#)
11. "Predictive Maintenance of Agricultural Equipment Using AI." (2023). *ITDS*. [Journals Enfoundations](#)
12. Lüttenberg, B., Bartelheimer, P. (Year not specified). Designing Predictive Maintenance for Agricultural Machines. Semantic Scholar. [Semantic Scholar](#)
13. "Enhancing Predictive Maintenance in Smart Agriculture ..." (2025). Zenodo record. [Zenodo](#)
14. Ahmed, M. T., et al. (2025). A systematic review of explainable artificial intelligence for (application domain). *ScienceDirect*. [ScienceDirect](#)

15. Sellam, B. B., & Arunachalam, R. (2020). Concerning use of ML & XAI in agriculture. *(Cited within Enhancing Predictive Maintenance paper)* [Zenodo](#)
16. (Thompson) Igunma, T. O., & Etukudoh, E. A. (2025). Utilizing Advanced Analytics and Machine Learning for Predictive Maintenance of Industrial Equipment. [ScienceDirect+1](#)
17. “Legal framework and data storage background of online collected data for predictive maintenance ... in the automotive sector in the European Union.” (2025). (Relevant for regulatory & privacy considerations in XAI/PdM) [ijssat.org+1](#)
18. “Human-in-the-Loop XAI for Predictive Maintenance: A Systematic Review of Interactive Systems and Their Effectiveness in Maintenance Decision-Making.” (2025). [ijssat.org+1](#)
19. “Predictive maintenance practices in manufacturing: systematic review.” Benhanifia, A. (2025). *ScienceDirect*. [ScienceDirect](#)
20. Murindanyi, S., Nakatumba-Nabende, J., Sanya, R., Nakibuule, R., & Katumba, A. (2024). Enhanced Infield Agriculture with Interpretable Machine Learning Approaches for Crop Classification. arXiv.